

claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-a6e3e9c853b8f897b.jsonl`
- SHA-256: `64650d1f9a67596e4c89a218a3bdf605cf8b82a959d5cc18d0d7eaf653ad930c`
- Source modified: `2026-06-20T09:00:58+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`
```

Transcript

1. user (2026-06-20T08:57:11.750Z)

Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master. The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the worker.

Files changed/added (read them in the WORKING TREE ? changes are uncommitted):

```
- backend/pipeline/detectors/native_wasb_runner.py (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py (added --device, threaded device into
_build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py (_resolve_runner: new 'native' branch +
_build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py (_build_native_runner override; WASB
substrate only)
- backend/config/models.py (WasbIndexConfig.device + .python_bin;
detector_impl doc mentions 'native')
- backend/worker/__main__.py (cmd_train: --trajectory-source
{synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs
runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)
```

DESIGN INTENT / CONTRACTS (judge against these):

```
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner,
WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively
(no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path
translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename
'{stem}__{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) ->
identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra
overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory
source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-
platform path handling matters.
- weights/repo/python/scratch come from env first
(WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.
```

YOU ARE THE WORKER REAL-EXTRACT SKEPTIC. Read backend/worker/__main__.py cmd_train + _resolve_train_detector. Find correctness bugs in the detector path: label<->video matching by stem (what if a label has no video, or a video has no label, or stems collide, or extensions differ), fps/frame_width probing on the real video (is _probe_fps_width correct here; what if cv2 can't open it), the >=2 LOVO count check (pre-loop on labels AND post-loop on specs), healthcheck gating, error/skip handling, and that the DEFAULT synth path is byte-unchanged from before. Also: is 'backend importing training' still avoided? Does _resolve_train_detector passing config (pydantic AppConfig) into seg_cls(None, config) work given segmenters call

```
self.config.get(...)?
```

2. assistant (2026-06-20T08:57:14.261Z)

I'll start by pulling the latest and reading the changed files in the working tree, focusing on the worker detector path.

3. user (2026-06-20T08:57:19.670Z)

```
M backend/config/models.py
M backend/pipeline/detectors/wasb_infer.py
M backend/pipeline/segmenters/fusion_hybrid.py
M backend/pipeline/segmenters/trajectory_hybrid.py
M backend/pipeline/segmenters/wasb_hybrid.py
M backend/worker/__main__.py
M tests/test_worker.py
?? artifacts/
?? backend/pipeline/detectors/native_wasb_runner.py
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
?? tests/test_native_wasb_runner.py
---BRANCH---
feat/native-wasb-runner
---LOG---
5c77f48 docs(decoupled): HANDOFF.md ? session checkpoint / resume-here (#255)
d5407eb feat(worker): Gen-0 train-from-bucket (worker train) on a CPU stub-trajectory mock
(#254)
814d813 feat(deploy): GPU machine provisioning ? NVMe scratch + native WASB setup (#253)
```

4. user (2026-06-20T08:57:19.680Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import csv
24     import json
25     import os
26     import sys
27     import tempfile
28     from typing import Any, List
29
30     from backend import storage
31     from backend.config import load_config
32     from backend.infrastructure.database import Database
33     from backend.pipeline.segmenters.base import segmenter_registry
34     from backend.pipeline.validators.base import registry as validator_registry
```

```

31     from backend.reporting import emit_indexer_report
32     from backend.results import Failure
33     from backend.utils.hashing import compute_video_id
34
35
36     def _coerce(v: str) -> Any:
37         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
38         str."""
39         low = v.lower()
40         if low in ("true", "false"):
41             return low == "true"
42         for cast in (int, float):
43             try:
44                 return cast(v)
45             except ValueError:
46                 pass
47         return v
48
49     def _apply_overrides(config: Any, sets: List[str]) -> None:
50         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
51         indexing.skip_ai_handoff=true)."""
52         for kv in sets or []:
53             key, sep, val = kv.partition("=")
54             if not sep:
55                 raise ValueError(f"--set expects k=v, got {kv!r}")
56             parts = key.split(".")
57             obj = config
58             for p in parts[:-1]:
59                 obj = getattr(obj, p)
60             setattr(obj, parts[-1], _coerce(val))
61
62     def _write_intervals_csv(segments: List[dict], path: str) -> None:
63         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
64         friendly artifact (same schema as the rally-annotator labels)."""
65         with open(path, "w", newline="") as f:
66             w = csv.writer(f)
67             w.writerow(["start", "end", "shot_count", "ending_reason"])
68             for s in segments:
69                 w.writerow([
70                     f'{float(s.get("start_time", 0.0)):.3f}',
71                     f'{float(s.get("end_time", 0.0)):.3f}',
72                     s.get("shot_count", ""),
73                     s.get("ending_reason", s.get("shot_count_confidence", "")),
74                 ])
75
76
77     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
78     None:
79         with open(path, "w") as f:
80             json.dump({
81                 "video_id": video_id,
82                 "status": status,
83                 "segment_count": len(segments),
84                 "segments": [{"start": float(s.get("start_time", 0.0)),
85                             "end": float(s.get("end_time", 0.0))} for s in segments],
86             }, f, indent=2)
87
88     def cmd_infer(args: argparse.Namespace) -> int:
89         config = load_config(args.config) if args.config else load_config()
90         _apply_overrides(config, args.set)
91         storage_cfg = config.storage.model_dump()

```

```

92
93     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
94     os.makedirs(scratch, exist_ok=True)
95     config.output.output_dir = scratch
96
97     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
98     local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
99     print(f"[worker] pulled {args.video_uri} -> {local_video}")
100
101     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
102     video_id = compute_video_id(local_video)
103
104     # 3. VALIDATE (same registry as the main CLI).
105     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
106     failures = [r for r in val_results if not r.passed]
107     db = Database(os.path.join(scratch, config.output.db_name))
108     db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
109
110
111     segments: List[dict] = []
112     segmenter_actual = None
113     segmentation_ran = False
114     status = "failed"
115     try:
116         if failures:
117             print("[worker] validation FAILED: "
+ "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
118             return 2
119
120             seg_name = args.segmenter or config.indexing.default_segmenter
121             segmenter_cls = segmenter_registry.get(seg_name)
122             if not segmenter_cls:
123                 print(f"[worker] unknown segmenter: {seg_name!r} "
f"(have {list(segmenter_registry.list_available())})")
124                 return 2
125
126             segmenter_actual = seg_name
127             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
128             segmentation_ran = True
129             if isinstance(result, Failure):
130                 print(f"[worker] segmenter FAILED: {result.message}")
131                 return 3
132             segments = result.value if hasattr(result, "value") else result
133             status = "success"
134     finally:
135         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
136         emit_indexer_report(
137             db=db, video_id=video_id, video_path=local_video, config=config,
138             val_results=val_results, val_context=val_context,
139             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
140             was_reingest=False, segmentation_ran=segmentation_ran,
141         )
142
143     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
144     out_local = os.path.join(scratch, "outputs", video_id)
145     os.makedirs(out_local, exist_ok=True)
146     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
147     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
148
149     out_prefix = args.out_uri.rstrip("/")

```

```

150     pushed = []
151     for fn in sorted(os.listdir(out_local)):
152         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154         pushed.append(uri)
155
156     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
157     for u in pushed:
158         print("    ", u)
159     return 0
160
161
162 def _resolve_train_detector(args: argparse.Namespace, config: Any):
163     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
164     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
165     stub=CPU mock). Returns the runner, or prints an error + returns None.
166
167     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
168     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
169     no shuttle detector and is rejected.
170     """
171     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
172
173     seg_cls = segmenter_registry.get(args.segmenter)
174     if not seg_cls:
175         print(f"[worker] unknown segmenter: {args.segmenter!r} "
176               f"(have {list(segmenter_registry.list_available())})")
177         return None
178     seg = seg_cls(None, config)
179     if not isinstance(seg, TrajectoryHybridSegmenter):
180         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
181               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
182         return None
183     runner, _ = seg._resolve_runner(config.get("indexing", {}))
184     ok, msg = runner.healthcheck()
185     if not ok:
186         print(f"[worker] detector environment not ready: {msg}")
187         return None
188     print(f"[worker] detector ready ({args.segmenter}): {msg}")
189     return runner
190
191
192 def cmd_train(args: argparse.Namespace) -> int:
193     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
194     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
195
196     Two trajectory sources (the train pipeline + harness are identical either way):
197     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
198     shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
199     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
200     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
201     is the M3.5 "first real training" path; only the trajectory source flips.
202     """
203     import subprocess
204

```

```

205     config = load_config(args.config) if args.config else load_config()
206     _apply_overrides(config, args.set)
207     storage_cfg = config.storage.model_dump()
208
209     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
210     labels_dir = os.path.join(scratch, "labels")
211     traj_dir = os.path.join(scratch, "trajectories")
212     videos_dir = os.path.join(scratch, "videos")
213     os.makedirs(labels_dir, exist_ok=True)
214     os.makedirs(traj_dir, exist_ok=True)
215
216     corpus = args.corpus_uri.rstrip("/")
217     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
218                        if u.lower().endswith(".csv"))
219     if len(label_uris) < 2:
220         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
221              f"under {corpus}/labels/")
222         return 2
223
224     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
225     runner = None
226     video_by_stem: dict = {}
227     if args.trajectory_source == "detector":
228         os.makedirs(videos_dir, exist_ok=True)
229         runner = _resolve_train_detector(args, config)
230         if runner is None:
231             return 2
232         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
233             vname = storage.parse_uri(vuri).name
234             video_by_stem[os.path.splitext(vname)[0]] = vuri
235
236     print(f"[worker] trajectory source: {args.trajectory_source}")
237     specs: List[dict] = []
238     for uri in label_uris:
239         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
240         name = fname.replace(".rallies.csv", "").replace(".csv", "")
241         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
242         if args.trajectory_source == "detector":
243             vuri = video_by_stem.get(name)
244             if not vuri:
245                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
246                 continue
247             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
248                                     config=storage_cfg)
249             video_id = compute_video_id(local_video)
250             # Real fps/width drive the trajectory frame->time mapping (synth fixes them
at 30/1280).
251             from backend.pipeline.segmenters.trajectory_hybrid import
TrajectoryHybridSegmenter
252             fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
253             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
254             if not traj:
255                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
256                 continue
257             specs.append({"name": name, "trajectory": traj, "labels": local_label,
258                          "fps": fps, "frame_width": frame_width})
259         else:

```

```

260         from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
261         traj = os.path.join(traj_dir, f"{name}_traj.csv")
262         synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
263         specs.append({"name": name, "trajectory": traj, "labels": local_label,
264                      "fps": args.fps, "frame_width": args.frame_width})
265
266     if len(specs) < 2:
267         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}].")
268         return 2
269     manifest_path = os.path.join(scratch, "manifest.json")
270     with open(manifest_path, "w", encoding="utf-8") as f:
271         json.dump(specs, f, indent=2)
272     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
273     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
274
275     out_dir = os.path.join(scratch, "artifacts")
276     cmd = [sys.executable, "-m", "training.gen0.harness",
277           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
278     if args.floor:
279         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
280                        if "://" in args.floor else args.floor)
281         cmd += ["--floor", floor_local]
282     print("[worker] training:", " ".join(cmd))
283     rc = subprocess.run(cmd).returncode
284     if rc != 0:
285         print(f"[worker] gen0 harness failed (rc={rc})")
286         return rc
287
288     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
289     bundle = os.path.join(out_dir, args.version)
290     out_prefix = args.out_uri.rstrip("/")
291     pushed = []
292     for fn in sorted(os.listdir(bundle)):
293         uri = f"{out_prefix}/weights/{args.version}/{fn}"
294         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
295         pushed.append(uri)
296     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
297     for u in pushed:
298         print("    ", u)
299     return 0
300
301
302 def build_parser() -> argparse.ArgumentParser:
303     p = argparse.ArgumentParser(prog="python -m backend.worker",
304                                description="Storage-aware worker: pull ? run pipeline ?
push.")
305     sub = p.add_subparsers(dest="cmd", required=True)
306
307     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
308     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
309     pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
310     pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
311     pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
312     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
313     pi.add_argument("--max-frames", type=int, default=None)

```

```

314         pi.add_argument("--set", action="append", default=[], metavar="k=v",
315                           help="config override, e.g. --set indexing.skip_ai_handoff=true")
316         pi.set_defaults(func=cmd_infer)
317
318         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
319         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
320                           "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
321         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
322         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
323         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
324                           help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
325                           "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
326                           "(needs videos/ in the corpus; honours detector_impl).")
327         pt.add_argument("--segmenter", default="wasb_hybrid",
328                           help="trajectory hybrid whose detector is used for --trajectory-
source detector")
329         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
330         pt.add_argument("--config", default=None)
331         pt.add_argument("--scratch", default=None)
332         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
333         pt.add_argument("--frame-width", type=float, default=1280.0,
334                           help="synth trajectory frame width (detector probes the video)")
335         pt.add_argument("--set", action="append", default=[], metavar="k=v")
336         pt.set_defaults(func=cmd_train)
337         return p
338
339
340     def main(argv=None) -> int:
341         args = build_parser().parse_args(argv)
342         return args.func(args)
343
344
345     if __name__ == "__main__":
346         sys.exit(main())
347

```

5. assistant (2026-06-20T08:57:22.542Z)

Let me read the related files to understand the contracts: the runner's
`run\_predict`/`healthcheck` signatures, `\_resolve\_runner`, `\_probe\_fps\_width`, and the new
tests.

6. user (2026-06-20T08:57:23.969Z)

```

1      """Shared engine for trajectory-substrate hybrid segmenters.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
6
7      healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8      -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family`` ? config section under ``indexing.*``, output subdir, and
12       the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").

```



```

14 - ``name`` / ``description`` properties (the registry contract).
15 - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16     detection_params extras such as weights_path).
17
18 This base is deliberately NOT registered; only concrete subclasses register.
19 """
20 import os
21 import time
22 import logging
23 import concurrent.futures
24 from typing import Any, Dict, List, Optional, Tuple
25
26 import cv2
27
28 from backend.pipeline.segmenters.base import BasePlaySegmenter
29 from backend.utils.video import get_video_duration, extract_video_chunk
30 from backend.sports import sport_registry
31 from backend.providers import provider_registry
32 from backend.pipeline.detectors.base import DetectorRunner
33
34 logger = logging.getLogger(__name__)
35
36
37 def _fmt_ts(seconds: float) -> str:
38     seconds = max(0, int(seconds))
39     h, rem = divmod(seconds, 3600)
40     m, s = divmod(rem, 60)
41     return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44 class TrajectoryHybridSegmenter(BasePlaySegmenter):
45     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46 boundaries."""
47     #: config section under `indexing` (velocity_threshold lives there), the
48     #: output subdir for trajectories, and the metadata detector-tag prefix.
49     family: str = ""
50     #: human-readable label used in log lines.
51     display_label: str = ""
52
53     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
54 Any]]:
55         """Return (runner, detector-specific detection_params extras) for the default
56 (WSL) path."""
57         raise NotImplementedError
58
59     def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
60 Dict[str, Any]]:
61         """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
62 with a
63 native runner override this; the base refuses with a clear message (M3.5: WASB
64 only)."""
65         raise NotImplementedError(
66             f"detector_impl='native' is not supported for the "
67             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
68 has a "
69             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
70
71     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
72 Dict[str, Any]]:
73         """Resolve the detector runner, honouring the CPU-stub and Linux-native
74 overrides.
75
76         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
77         ``indexing.detector_impl``:

```

```

69         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73         (the WSL runner) so the default production path is unchanged
74         (``docs/DECOUPLED_COMPUTE/``).
75         """
76         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77         if impl == "stub":
78             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                         self.display_label or "trajectory")
81             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82         if impl in ("native", "native_wasb", "wasb_native"):
83             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                         self.display_label or "trajectory")
85             return self._build_native_runner(idx_cfg)
86         return self._build_runner(idx_cfg)
87
88     @staticmethod
89     def _probe_fps_width(video_path: str) -> tuple:
90         """Return (fps, frame_width) for a video, with sensible fallbacks."""
91         cap = cv2.VideoCapture(video_path)
92         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94         cap.release()
95         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97     @staticmethod
98     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99         """Provider-reported exchange count, else a duration-based estimate.
100
101         One canonical implementation ? the twins had drifted (wasb relied on
102         ``int(None)`` raising into the fallback; same result, by accident).
103         """
104         shot_count = segment.get("shuttle_exchange_count")
105         if shot_count is None:
106             return max(3, int(duration / 0.8))
107         try:
108             return int(shot_count)
109         except (ValueError, TypeError):
110             return max(3, int(duration / 0.8))
111
112     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-----
113     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
114                                frame_width: float, video_path: str,
115                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
117         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
119         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
120         return {
121             "peak_velocity": cw["peak_velocity"],
122             "mean_velocity": cw["mean_velocity"],
123             "active_frame_count": cw["active_frame_count"],

```

```

124         "track_id_count": cw["track_id_count"],
125     }
126
127     def _select_for_handoff(self, windows: List[Dict[str, Any]],
128                             window_stats: List[Dict[str, Any]],
129                             idx_cfg: Dict[str, Any]) -> List[int]:
130         """Indices of the candidate windows that proceed to the AI handoff (and thus may
131         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
132         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
133         Persisted candidates are NOT affected ? they remain the full superset for
offline
134         re-scoring."""
135         return list(range(len(windows)))
136
137     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
138                      player_pool: Optional[List[str]] = None,
139                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
140         # override-ignore: the ABC declares the Result contract (Success|Failure)
141         # but every live segmenter still returns a bare list ? the documented
142         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
143         label = self.display_label
144         # --- Sport profile + AI provider wiring (identical across segmenters) ---
145         sport_name = self.config.get("sport", "badminton")
146         try:
147             sport_profile = sport_registry.get(sport_name)
148         except KeyError as e:
149             logger.error(str(e))
150             return []
151
152         idx_cfg = self.config.get("indexing", {})
153         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
154         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
155         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
156         # measure it against the Gemini path, or to avoid the cloud entirely). With
157         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
158         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
159         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
160         if skip_ai:
161             model_name = "local-noai"
162             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
163                         "be emitted as rallies; no %s handoff, no API key needed.",
164                         label, provider_name)
165         else:
166             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
167             api_key_env_var = f"{provider_name.upper()}_API_KEY"
168             api_key = os.environ.get(api_key_env_var)
169             if not api_key:
170                 from backend.pipeline.segmenters._keyhint import api_key_hint
171                 logger.error(
172                     f"{api_key_env_var} environment variable is not set! "
173                     f"To run the {provider_name} handoff, set your API key. "
174                     + api_key_hint(api_key_env_var)
175                 )
176             return []
177         try:
178             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
179         except KeyError as e:

```

```

180         logger.error(str(e))
181         return []
182
183     video_duration = get_video_duration(video_path)
184     if video_duration <= 0:
185         logger.error("Invalid video duration.")
186         return []
187     fps, frame_width = self._probe_fps_width(video_path)
188
189     # --- Runner config + windowing thresholds ---
190     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
192     # it delegates to the subclass _build_runner (the real WSL/native runner).
193     runner, runner_params = self._resolve_runner(idx_cfg)
194
195     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
196     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
197     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
198     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
199     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
200     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
201     window_hard_cap = idx_cfg.get("window_hard_cap", False)
202     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
203     inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
204     inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
205     window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
206     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
207     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
208     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
209     log_candidates = idx_cfg.get("log_yolo_candidates", True)
210     store_candidates = idx_cfg.get("store_yolo_candidates", True)
211
212     detection_params = {
213         "detector": self.name,
214         "velocity_thresh": velocity_thresh,
215         "merge_gap": merge_gap,
216         "min_action_window_duration": min_window_duration,
217         **runner_params,
218     }
219
220     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
221     ok, msg = runner.healthcheck()
222     if not ok:
223         logger.error("%s detector environment not ready: %s", label, msg)
224         return []
225     logger.info("%s detector env OK: %s", label, msg)
226
227     # --- Phase 2: trajectory -> candidate rally windows ---
228     # Trajectory detectors process the whole video in one pass (they carry
229     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
230     t_start = time.time()
231     out_dir = os.path.join("output", self.family)
232     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
233     if not csv_path:
234         logger.error("%s produced no trajectory; aborting.", label)
235         return []
236
237     points = runner.parse_trajectory_csv(csv_path)
238     logger.info("Parsed %d trajectory points (%d visible).",
239                 len(points), sum(1 for p in points if p.visible))
240
241     all_candidate_windows = runner.trajectory_to_action_windows(
242         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
243         velocity_thresh=velocity_thresh, merge_gap=merge_gap,

```

```

244         min_window_duration=min_window_duration, chunk_index=0,
245         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
246         window_hard_cap=window_hard_cap,
247         window_inactivity_end=window_inactivity_end,
248         inactivity_rally_thresh=inactivity_rally_thresh,
249         inactivity_min_density=inactivity_min_density,
250         window_blob_fallback=window_blob_fallback,
251         window_blob_factor=window_blob_factor,
252     )
253     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
254               label, time.time() - t_start, len(all_candidate_windows))
255
256     if log_candidates:
257         for cw in all_candidate_windows:
258             logger.info(f"[{label}] rally]
259             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
260             f"({cw['end'] - cw['start']:.1f}s) |
261             frames={cw['active_frame_count']} "
262             f"peak_v={cw['peak_velocity']:.3f}
263             mean_v={cw['mean_velocity']:.3f}")
264
265     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
266     fusion
267     # segmenter adds net_crossings + person-cue features. Computed once, reused for
268     both
269     # persistence and the handoff gate below.
270     window_stats = [
271         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
272         idx_cfg)
273         for cw in all_candidate_windows
274     ]
275
276     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
277     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
278     if store_candidates:
279         for i, cw in enumerate(all_candidate_windows):
280             candidate_ids[i] = self.db.add_candidate_segment(
281                 video_id, detector=self.name,
282                 start_time=cw["start"], end_time=cw["end"],
283                 chunk_index=cw["chunk_index"], confidence=min(1.0,
284                 cw["peak_velocity"]),
285                 stats=window_stats[i], detection_params=detection_params,
286             )
287
288     if not all_candidate_windows:
289         return []
290
291     # --- Phase 3: AI provider handoff over padded candidate clips ---
292     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
293     Persisted
294     # candidates above stay the full superset ? only the served play_segments are
295     gated.
296     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
297     idx_cfg)
298
299     ai_segments: List[dict] = []
300     if skip_ai:
301         # Fully-local: the selected candidate windows ARE the rallies ? emit them
302         verbatim
303         # (no clip extraction, no upload). Boundaries are the local detector's; shot
304         count
305         # falls back to the duration-based estimate (no AI exchange count).
306         ai_segments = [
307             {
308                 "start_time": all_candidate_windows[wi]["start"],

```

```

297         "end_time": all_candidate_windows[wi]["end"],
298         "shuttle_exchange_count": None,
299         "shot_count_confidence": "LocalNoAI",
300         "_candidate_id": candidate_ids[wi],
301     }
302     for wi in handoff_indices
303 ]
304 logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
305             "(no AI handoff).", len(ai_segments))
306 else:
307     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
308     os.makedirs(handoff_dir, exist_ok=True)
309     logger.info("Phase 3: %s validation on %d candidate windows...",
310               provider_name, len(handoff_indices))
311
312     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
313         w_start = max(0, window["start"] - handoff_padding)
314         w_end = min(video_duration, window["end"] + handoff_padding)
315         w_dur = w_end - w_start
316         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
317         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
318             return []
319         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
320         segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
321                                             label=f"Handoff {wi+1}")
322         valid = []
323         for seg in segments:
324             try:
325                 seg["start_time"] = float(seg["start_time"]) + w_start
326                 seg["end_time"] = float(seg["end_time"]) + w_start
327                 seg["_candidate_id"] = candidate_ids[wi]
328                 valid.append(seg)
329             except (ValueError, TypeError, KeyError) as e:
330                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
331             try:
332                 os.remove(w_path)
333             except OSError:
334                 pass
335         return valid
336
337     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
338         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
339                    for wi in handoff_indices]
340         for future in futures:
341             ai_segments.extend(future.result())
342         try:
343             os.rmdir(handoff_dir)
344         except OSError:
345             pass
346         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
347
348     # --- Phase 4: DB population + candidate linking ---
349     segments_data = []
350     for segment in ai_segments:
351         try:
352             start = float(segment["start_time"])
353             end = float(segment["end_time"])
354             dur = end - start

```

```

355         metadata = {
356             "shot_count": self._shot_count_from(segment, dur),
357             "shot_count_confidence": segment.get("shot_count_confidence",
358             "Unknown"),
359             "detector": f"{self.family}-hybrid-{model_name}",
360         }
361         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
362         metadata)
363         if segment_id:
364             candidate_id = segment.get("_candidate_id")
365             if candidate_id is not None:
366                 self.db.link_candidate_to_segment(candidate_id, segment_id)
367                 segments_data.append({
368                     "id": segment_id, "start_time": start, "end_time": end,
369                     "duration": dur, "metadata": metadata,
370                 })
371         except Exception as e:
372             logger.warning("Skipping invalid segment struct: %s", e)
373
374     return segments_data
375

```

7. user (2026-06-20T08:57:24.207Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
5     M3.5).
6     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
7     Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
8     core
9     that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
10    transport:
11
12    * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
13    ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
14    (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
15    * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
16    into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
17    * weights / repo / python / scratch come from the **environment** first
18    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
19    ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
20
21    It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}.wasb.csv``
22    name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
23    ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
24    the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
25    (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
26    ``detector_impl="native"``.
27
28    **Acceptance gate (owner / GPU box):** byte-parity ? the SAME clip through the WSL
29    runner
30    (Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV
31    and
32    identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
33    env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
34    test suite.
35    """
36    import os
37    import shutil
38    import logging
39    import subprocess
40    from dataclasses import dataclass
41    from typing import Any, Dict, List, Optional, Tuple

```

```

37
38     from backend.pipeline.detectors.base import DetectorRunner
39     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
40     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
41
42     logger = logging.getLogger(__name__)
43
44     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
45     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
46     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
47
48
49     @dataclass
50     class NativeWasbConfig:
51         """Resolved settings for a Linux-native WASB run.
52
53         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
54         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
55         ``device``
56         defaults to ``cuda`` (the WSL byte-parity path).
57         """
58         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
59         python_bin: str = "python" # the `wasb` conda env python (invoked by
path, no activate)
60         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
61         sport: str = "badminton"
62         device: str = "cuda" # cuda | mps | cpu
63         scratch_dir: str = "" # frame-extraction scratch (env); "" = next
to the output dir
64         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no
timeout)
65         keep_frames: bool = False # retain the decoded frame cache after
success (debug only)
66         stream_video: bool = False # fast path: decode in-process, no PNG
extraction (bit-identical)
67
68     @classmethod
69     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
70         w = (idx_cfg or {}).get("wasb", {}) or {}
71         env = os.environ.get
72         return cls(
73             repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
74             python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
75             weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
76             sport=w.get("sport", "badminton"),
77             device=env("WASB_DEVICE") or w.get("device", "cuda"),
78             scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
"",
79             timeout_sec=int(w.get("timeout_sec", 1800)),
80             keep_frames=bool(w.get("keep_frames", False)),
81             stream_video=bool(w.get("stream_video", False)),
82         )
83
84     class NativeWasbRunner(DetectorRunner):
85         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
86
87         def __init__(self, cfg: NativeWasbConfig):
88             self.cfg = cfg
89
90         # --- low-level native exec (the single place tests patch)
-----

```



```

91         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
92             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
93             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
94                                   timeout=(self.cfg.timeout_sec or None))
95
96         def _repo_src(self) -> str:
97             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
98
99         def _copy_infer_script(self) -> bool:
100             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
101             repo_src = self._repo_src()
102             try:
103                 os.makedirs(repo_src, exist_ok=True)
104                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
105                 return True
106             except OSError as e:
107                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
108                 return False
109
110         def _rm_rf(self, path: Optional[str]) -> None:
111             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
112             if path:
113                 shutil.rmtree(path, ignore_errors=True)
114
115         def healthcheck(self) -> Tuple[bool, str]:
116             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
117             if not self.cfg.weights_path:
118                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
119             weights = os.path.expanduser(self.cfg.weights_path)
120             if not os.path.exists(weights):
121                 return False, f"WASB weights not found at {weights}"
122             repo_src = self._repo_src()
123             if not os.path.isdir(repo_src):
124                 return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
125                                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
126             code = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
127             try:
128                 res = self._run([self.cfg.python_bin, "-c", code])
129             except FileNotFoundError:
130                 return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
131             except subprocess.TimeoutExpired:
132                 return False, "native torch healthcheck timed out."
133             out = (res.stdout or "").strip()
134             if res.returncode != 0:
135                 return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
136             if self.cfg.device == "cuda" and "cuda True" not in out:
137                 return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
138             return True, out
139
140         def run_predict(self, video_win_path: str, output_win_dir: str,
141                        video_id: Optional[str] = None) -> Optional[str]:
142             """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
143
144             Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL

```

```

145         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
146         same downstream contract, so the parity comparison is apples-to-apples.
147         """
148         output_dir = os.path.abspath(output_win_dir)
149         os.makedirs(output_dir, exist_ok=True)
150         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
151         norm = str(video_win_path).replace("\\", "/")
152         stem, _ext = os.path.splitext(os.path.basename(norm))
153         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
154         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
155         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
156
157         if not self._copy_infer_script():
158             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
159             return None
160
161         weights = os.path.expanduser(self.cfg.weights_path)
162         argv = [self.cfg.python_bin, "wasb_infer.py",
163               "--video", os.path.abspath(str(video_win_path))]
164         frames_dir: Optional[str] = None
165         if self.cfg.stream_video:
166             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
167             argv.append("--stream-video")
168         else:
169             scratch = self.cfg.scratch_dir or output_dir
170             frames_dir = os.path.join(scratch, f"{key}_frames")
171             argv += ["--frames_out_dir", frames_dir]
172         argv += ["--weights", weights,
173               "--sport", self.cfg.sport,
174               "--device", self.cfg.device,
175               "--out", out_csv]
176
177         logger.info("Running native WASB inference (device=%s) on %s ...",
178                   self.cfg.device, os.path.basename(norm))
179         try:
180             res = self._run(argv, cwd=self._repo_src())
181         except FileNotFoundError:
182             logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
183             return None
184         except subprocess.TimeoutExpired:
185             logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
186             return None
187         if res.returncode != 0:
188             logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
189             return None
190         if not os.path.exists(out_csv):
191             logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
192             return None
193         logger.info("WASB trajectory CSV: %s", out_csv)
194
195         # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
196         # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
197         # (Streaming never materializes frames, so there is nothing to drop on that
path.)
198         if not self.cfg.keep_frames and frames_dir:
199             logger.info("Cache hygiene: removing frame cache for %s "

```

```

200             "(set indexing.wasb.keep_frames=true to retain).", key)
201         self._rm_rf(frames_dir)
202         return out_csv
203
204         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
205         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
206         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
207

```

8. assistant (2026-06-20T08:57:29.141Z)

Now let me look at the key contract details: `run\_predict` vs `run\_detector`/`run` naming, the `seg.\_resolve\_runner` call with `config.get(...)` , and how config is accessed. There's a critical signature mismatch I want to verify: `\_resolve\_train\_detector` calls `config.get("indexing", {})` where `config` is a pydantic `AppConfig`. Let me check.

9. user (2026-06-20T08:57:30.119Z)

```

1         """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3         This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4         of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5         the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7         The ABC lets us:
8         - Keep the current WSL adapters working unchanged for Windows users.
9         - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11        - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14        Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15        - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16          Run detector, return the Windows-side path to the produced trajectory CSV
17          (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18          The caller (hybrid) is responsible for output dir; impls may stage the video
19          into WSL fs for perf.
20
21        - healthcheck() -> tuple[bool, str]
22          Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23          Message is logged (and can be surfaced to reports or UX in future).
24          Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26        Existing behavior is 100% preserved. The only additions are the inheritance and
27        the explicit common type for future implementers.
28
29        How to add a new runner (for docs / future contributors):
30        1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31        2. Implement run_predict and healthcheck (return (bool, str) tuple).
32        3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33          TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34          deliberately static and model-agnostic). If the new detector needs different
35          windowing, it can implement its own or we generalize later.
36        4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37          registry never pulls torch/tf on a machine that doesn't have them).
38        5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39          DetectorRunner instances (they already do the right thing via duck-typing + the
40          type hints we add here).
41        6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42          returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43        7. No change to eval/calibration code paths (they consume the CSV + the static
44          windowing helpers directly).
45

```

```

46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 """
68
69 import logging
70 import shlex
71 import subprocess
72 from abc import ABC, abstractmethod
73 from typing import Any, Callable, List, Optional, Tuple
74
75 logger = logging.getLogger(__name__)
76
77
78 def wsl_tilde_quote(path: str) -> str:
79     """shlex.quote that still lets bash expand a leading `~/`.
80
81     ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
82     treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
83     directory" ? this broke live WSL staging for the default ``~/clips``
84     stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85     "shlex-quote breaks ~ expansion" item). Quoting only the part after
86     ``~/`` keeps expansion AND protects spaces/specials in the remainder.
87     """
88     if path == "~":
89         return path
90     if path.startswith("~/"):
91         return "~/ " + shlex.quote(path[2:])
92     return shlex.quote(path)
93
94
95 class WslCommandMixin:
96     """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
97
98     Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99     inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
100     and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
101     ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
102     """
103
104     cfg: Any
105
106     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109         logger.debug("WSL exec: %s", full)
110         return subprocess.run(argv, capture_output=True, text=True,

```

```

111                 timeout=(self.cfg.timeout_sec or None))
112
113     # ---- cache hygiene helpers (shared by both WSL runners) -----
114     def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115         """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.
116
117         Returns None if it can't be determined (never raises). Used as a pre-run
118         disk guard so a long extraction doesn't fill the disk mid-flight.
119         """
120         try:
121             res = self._wsl_bash(f'df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122 'NR==2{{print $4}}}')
123         except (subprocess.TimeoutExpired, OSError):
124             return None
125         try:
126             return int((res.stdout or "").strip()) / (1024 ** 3)
127         except (ValueError, AttributeError):
128             return None
129
130     def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
131         """Best-effort recursive delete of WSL paths (post-success cleanup; never
132 raises)."""
133         paths = [p for p in wsl_paths if p]
134         if not paths:
135             return
136         quoted = " ".join(wsl_tilde_quote(p) for p in paths)
137         try:
138             self._wsl_bash(f"rm -rf {quoted}")
139         except (subprocess.TimeoutExpired, OSError) as e:
140             logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
141
142     def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
143         """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
144
145         Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
146         filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
147         fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
148         trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
149         A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
150         any
151         frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
152         ``cp`` still gets its chance (we never block a genuinely-readable source).
153         """
154         try:
155             res = self._wsl_bash(f'test -r {shlex.quote(src_mnt)} && echo READABLE')
156         except (subprocess.TimeoutExpired, OSError):
157             return None
158         if "READABLE" in (res.stdout or ""):
159             return None
160         return (
161             f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
162             f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
163             f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
164         )
165
166     class DetectorRunner(ABC):
167         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
168
169         @abstractmethod
170         def run_predict(self, video_win_path: str, output_win_dir: str,
171             video_id: Optional[str] = None) -> Optional[str]:
172             """Run inference and return path to trajectory CSV (or None on failure).
173
174             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two

```

```

173         videos that happen to share a filename cannot reuse each other's frames or CSV.
174         It is optional for back-compat; callers in the live pipeline always pass it.
175         """
176         pass
177
178     @abstractmethod
179     def healthcheck(self) -> Tuple[bool, str]:
180         """Quick check if the runner (env, weights, etc.) is usable.
181
182         Returns (ok, message). Never raises. Cheap to call before a long run.
183         """
184         pass
185
186     # De facto trajectory contract: the trajectory hybrids call these
187     # unconditionally after run_predict. Concrete runners reuse/re-export the
188     # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189     # a runner with custom windowing supplies its own with the same shapes.
190     parse_trajectory_csv: Callable[..., Any]
191     trajectory_to_action_windows: Callable[..., Any]
192

```

10. user (2026-06-20T08:57:30.363Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""

```

```

41         try:
42             temp_inst = segmenter_cls(None, {})
43             name = temp_inst.name
44         except Exception as e:
45             raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46         self._segmenters[name] = segmenter_cls
47
48     def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49         return self._segmenters.get(name)
50
51     def list_available(self) -> Dict[str, str]:
52         """Returns a dict of registered segmenter names and their descriptions."""
53         res = {}
54         for name, cls in self._segmenters.items():
55             try:
56                 temp_inst = cls(None, {})
57                 desc = temp_inst.description
58             except Exception:
59                 desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

11. assistant (2026-06-20T08:57:34.994Z)

Now I've found a critical issue. The runner contract method is `run_predict`, but `cmd_train` calls `runner.run_predict(...)` ? let me re-check. Actually line 253 calls `runner.run_predict(local_video, traj_dir, video_id=video_id)`. That matches the ABC. Good.

But there's the `config.get(...)` question. `seg = seg_cls(None, config)` where `config` is the pydantic `AppConfig`, and then `seg._resolve_runner(config.get("indexing", {}))`. The segmenter's `self.config.get(...)` is used heavily. Let me verify what `load_config` returns and whether `AppConfig` supports `.get()`.

12. user (2026-06-20T08:57:35.618Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720

```

```

27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72         device: str = "cuda" # torch device for the native runner: cuda | mps | cpu
73         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
74         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
75         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
76         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
77         keep_frames: bool = False
78         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
79         min_free_gb: float = 0.0
80         # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
81         # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
82         # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
83         # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is

```



```

lossless),
84         # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
85         stream_video: bool = False
86
87
88     class FusionConfig(BaseModel):
89         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
90
91         Every default reproduces control behaviour: the fusion segmenter persists the
92         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
93         true ? the person-cue features, but it does NOT gate the served handoff unless a
94         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
95         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
96         supplied ? off-court persons (spectators / adjacent courts) are excluded.
97         """
98         # Which trajectory substrate seeds the windows (shuttle stays primary).
99         substrate: Literal["wasb", "tracknet"] = "wasb"
100        # Windowing velocity threshold for the fusion family (the engine reads
101        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
102        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
103        velocity_threshold: float = 0.02
104        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
105        # enabled ? so the default path never imports torch / touches the GPU.
106        cue_flags: dict[str, bool] = Field(default_factory=dict)
107        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
108        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
109        min_crossings: int = Field(default=0, ge=0)
110        # Served Gate B: when the person cue is on, drop windows with median in-court
players
111        # < this. 0 = OFF.
112        min_players: int = Field(default=0, ge=0)
113        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
114        court_polygon: Optional[list[list[float]]] = None
115        # Net line for the person two-sided-motion split (person features only ? the shuttle
116        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
117        net_axis: Literal["x", "y"] = "y"
118        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
119
120
121     class IndexingConfig(BaseModel):
122         default_segmenter: str = "yolo_hybrid"
123         use_gpu: bool = True
124         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
125         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
126         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
127         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
128         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
129         detector_impl: str = "auto"
130         motion_threshold: float = 0.08
131         min_segment_duration: float = 3.0
132         dead_time_merge_gap: float = 2.0
133         chunk_duration: float = 90.0
134         chunk_overlap: float = 10.0
135         detector_model: str = "fasterrcnn_resnet50_fpn"
136         detector_score_threshold: float = 0.5
137         yolo_velocity_threshold: float = 0.15
138         yolo_frame_skip: int = 3

```

```

139     yolo_tracker: str = "bytetrack.yaml"
140     yolo_prefetch_workers: int = 2
141     min_action_window_duration: float = 2.0
142     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
143     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
144     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
145     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
146     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
147     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
148     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
149     max_window_duration: float = 6.0
150     window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
151     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
152     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
153     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
154     # Only cuts (recall can't drop). OFF by default until measured/promoted.
155     window_hard_cap: bool = False
156     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
157     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
158     # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
159     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
160     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
161     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
162     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
163     window_inactivity_end: float = 1.5
164     inactivity_rally_thresh: float = 0.20
165     inactivity_min_density: float = 0.30
166     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
167     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
168     # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
169     # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
170     # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
171     # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
172     # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
173     # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
174     window_blob_fallback: bool = False
175     # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
176     # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).

```

```

177 # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
178 # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
179 window_blob_factor: float = 4.0
180 log_yolo_candidates: bool = True
181 store_yolo_candidates: bool = True
182 ai_handoff_padding: float = 2.0
183 ai_max_workers: int = 5
184 # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
185 # chunks of a high-bitrate (4K) source blow past the provider upload cap
186 # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
187 # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
188 # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
189 # Matches gemini_refine's --clip-scale-width default.
190 ai_chunk_scale_width: int = 1280
191 # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
192 # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
193 # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
194 # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
195 # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
196 skip_ai_handoff: bool = False
197 # Optional debug knob: trim every video to the first N seconds before processing.
198 debug_duration: Optional[float] = None
199
200 tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
201 wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
202 fusion: FusionConfig = Field(default_factory=FusionConfig)
203
204 @field_validator("default_segmenter")
205 @classmethod
206 def segmenter_must_be_registered(cls, v: str) -> str:
207     # Registry check happens at runtime in main/segmenter selection.
208     # Here we just allow any string for forward compatibility.
209     return v
210
211 @model_validator(mode="after")
212 def _chunk_step_must_be_positive(self) -> "IndexingConfig":
213     # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
214     # step makes `while t < duration: t += step` loop forever, growing memory before
any
215     # GPU work. Reject the bad config at load time instead.
216     if self.chunk_overlap >= self.chunk_duration:
217         raise ValueError(
218             f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
219             f"({self.chunk_duration}); otherwise chunking never advances."
220         )
221     return self
222
223
224 class OutputConfig(BaseModel):
225     default_highlights_name: str = "highlights.mp4"
226     db_name: str = "sports_indexer.db"
227     # Directory where the DB, highlight reels, and processed clips are written.
228     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
229     output_dir: str = "output"
230
231
232 class WatermarkConfig(BaseModel):

```

```

233     """Branding overlay burned into each highlight clip during the per-clip re-encode
234     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
235     (under `stitching.watermark` in config.json) to turn it off. The text is fully
236     configurable. The concat stage stays stream-copy (-c copy)."""
237     enabled: bool = True
238     text: str = "Generated by Khelsutra.guru"
239     logo_path: str = ""          # optional transparent PNG overlay; "" = no logo
240     font_path: str = ""          # optional .ttf; "" = use the fontconfig `font` family
below
241     font: str = "Sans"           # fontconfig family used when font_path is empty
242     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
243     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
244     box: bool = True             # faint backing box behind the text for legibility
245     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
246     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
247
248
249     class StitchingConfig(BaseModel):
250         begin_padding: float = 0.5
251         end_padding: float = 0.5
252         use_cache: bool = False
253         min_shot_count: int = 1
254         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
255         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
256         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
257         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
258         # local detector over-fires and its false positives are mostly SHORT (motion blips,
259         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
260         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
261         min_rally_duration: float = Field(default=0.0, ge=0.0)
262         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
263
264
265     class LoggingConfig(BaseModel):
266         """Logging knobs for the run endpoints (see backend/utils/logging_setup.py)."""
267         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
268         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
269         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
270         # them to WARNING; set true to restore full chatter for request-flow debugging.
271         verbose_http: bool = False
272
273
274     class SecurityConfig(BaseModel):
275         # When set, /api/process video_path must resolve under this directory (hosted/tenant
276         # mode). Default None preserves the single-user local 'any local file' workflow.
277         allowed_video_root: Optional[str] = None
278
279         # --- Chunked upload (B2, PR-2) ---
280         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
281         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
282         # contain upload_dir or /api/process will reject the uploaded paths.
283         upload_dir: str = "output/uploads"
284         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
285         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
286         max_upload_mb: int = 4096
287         max_part_mb: int = 95
288         allowed_upload_extensions: List[str] = ["mp4", "mov"]
289

```

```

290
291 class StorageConfig(BaseModel):
292     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
293     behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
294     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google
Drive
295     (follow-up). Per-backend settings are loose dicts passed to the backend constructor
?
296     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
297     (boto3/google auth chains supply credentials). See ``backend/storage/``. """
298     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
299     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
300     output_root: str = ""
301     local: dict[str, Any] = Field(default_factory=dict)
302     s3: dict[str, Any] = Field(default_factory=dict)
303     gcs: dict[str, Any] = Field(default_factory=dict)
304     gdrive: dict[str, Any] = Field(default_factory=dict)
305
306
307 class AppConfig(BaseModel):
308     """Root configuration object.
309
310     All runtime configuration should be accessed via an instance of this class
311     (or the loader) rather than raw dict lookups.
312     """
313
314     sport: Sport = "badminton"
315     ai_provider: str = "gemini"
316     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
317     # the alias never deprecates underneath us. Eval reports record the run date
alongside.
318     ai_model: str = "gemini-flash-latest"
319
320     validation: ValidationConfig = Field(default_factory=ValidationConfig)
321     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
322     output: OutputConfig = Field(default_factory=OutputConfig)
323     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
324     security: SecurityConfig = Field(default_factory=SecurityConfig)
325     logging: LoggingConfig = Field(default_factory=LoggingConfig)
326     storage: StorageConfig = Field(default_factory=StorageConfig)
327
328     # Allow extra keys during transition so existing config.json files do not break.
329     model_config = {
330         "extra": "allow",
331         "validate_assignment": True,
332     }
333
334     # Temporary migration shim for legacy dict-style access (config.get("section",
{})).get(...)).
335     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
336     # attribute access / the typed config. A nested *section* is returned as a plain
dict
337     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
338     # section model instead breaks it (a model has no .get), which bit every validator
on
339     # real runs. Scalar leaves are returned as-is.
340     def get(self, key: str, default: Any = None) -> Any:
341         _MISSING = object()
342         val: Any = _MISSING
343         if key in type(self).model_fields or hasattr(self, key):
344             val = getattr(self, key, _MISSING)

```

```

345         if val is _MISSING:
346             # Legacy convenience: find a leaf key inside any nested section. Sections
are
347             # discovered dynamically from the model fields, so a future top-level
section
348             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
349             for fname in type(self).model_fields:
350                 section = getattr(self, fname, None)
351                 if isinstance(section, BaseModel) and hasattr(section, key):
352                     val = getattr(section, key)
353                     break
354         if val is _MISSING:
355             return default
356         if isinstance(val, BaseModel):
357             return val.model_dump(mode="json")
358         return val
359
360     def __getitem__(self, key: str) -> Any:
361         val = self.get(key)
362         if val is None and key not in type(self).model_fields:
363             # Re-raise like a real dict for unknown top-level
364             raise KeyError(key)
365         return val
366
367
368     # Convenience type alias for callers that want to annotate
369     Config = AppConfig
370

```

13. assistant (2026-06-20T08:57:40.587Z)

Good. So `config.get("indexing", {})` returns `indexing` as a `**dict**` via `model_dump(mode="json")`, because `IndexingConfig` is a `BaseModel`. Now the important subtlety: `seg = seg_cls(None, config)` passes the full `AppConfig` (a model), but then `_resolve_runner(config.get("indexing", {}))` passes a `**dict**`. Let me verify `_resolve_runner` and `_build_native_runner` only use dict `.get(...)` on `idx_cfg`. From `trajectory_hybrid`, `_resolve_runner` uses `idx_cfg.get(...)` ? fine on a dict. Let me check the `wasb_hybrid` `_build_native_runner` override.

14. user (2026-06-20T08:57:41.996Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Phase 2 uses WASB (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4     WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5     badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7     For the per-video calibration goal we mainly use Phase 2 (trajectory ->
8     candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9     min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):

```

```

24     family = "wasb"
25     display_label = "WASB"
26
27     @property
28     def name(self) -> str:
29         return "wasb_hybrid"
30
31     @property
32     def description(self) -> str:
33         return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                 "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37         cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38         return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40     def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
41         # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
42         # seam (imported only when detector_impl='native' is actually selected).
43         from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
44         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
45         return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
46                                       "detector_impl": "native", "device": cfg.device}
47
48
49     segmenter_registry.register(WasbHybridSegmenter)
50

```

15. user (2026-06-20T08:57:42.278Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but

```

```

23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54         @property
55         def name(self) -> str:
56             return "fusion_hybrid"
57
58         @property
59         def description(self) -> str:
60             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                     "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                     "AI provider sets semantic boundaries.")
63
64         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66             if substrate == "tracknet":
67                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                 return TrackNetRunner(cfg), {
69                     "weights_path": cfg.weights_path,
70                     "queue_length": cfg.queue_length,
71                     "fusion_substrate": "tracknet",
72                 }
73             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,

```



```

Dict[str, Any]]:
77         # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
78         # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
79         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
80         if substrate != "wasb":
81             raise NotImplementedError(
82                 "detector_impl='native' supports the WASB substrate only ? set "
83                 "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
84         from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
85         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
86         return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
"detector_impl": "native",
87                                     "fusion_substrate": "wasb", "device": cfg.device}
88
89     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
90                               frame_width: float, video_path: str,
91                               idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
92         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
93         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
94         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
95         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
96         gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
97                                               min_crossings=0,
estimate=self._axis_estimate(points))
98         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
99
100         fcfg = idx_cfg.get("fusion", {})
101         if fcfg.get("cue_flags", {}).get("person", False):
102             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
103         return stats
104
105     # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
106     # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
107     # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
108     # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
109     # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
110     # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
111     # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
112
113     def _axis_estimate(self, points: List[Any]) -> tuple:
114         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
115         computed once per video, not once per candidate window (it depends only on
points)."""
116         key = id(points)
117         if self._axis_cache_key != key or self._axis_cache is None:
118             self._axis_cache_key = key
119             self._axis_cache = rally_gate.estimate_play_axis(points)
120         est = self._axis_cache
121         assert est is not None
122         return est

```

```

123
124     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
125                           frame_width: float, video_path: str,
126                           fcfg: Dict[str, Any]) -> Dict[str, float]:
127         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
128         frame range and compute the scale/translation-invariant person features. Lazily
builds
129         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
130         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
131         from backend.pipeline.detectors.person_detector import PersonDetector
132         if self._person is None:
133             self._person = PersonDetector(self.config)
134
135         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
136         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
137         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
138         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
139         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
140         fw = det["frame_width"] or frame_width
141         fh = det["frame_height"]
142         # If we can't trust the frame dims, every normalised feature would silently
collapse
143         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
144         if fw <= 0 or fh <= 0:
145             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
146
147         poly = fcfg.get("court_polygon")
148         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
149         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
150         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
151         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
152                                           court_polygon=polygon, net=net)
153
154     def _select_for_handoff(self, windows: List[Dict[str, Any]],
155                             window_stats: List[Dict[str, Any]],
156                             idx_cfg: Dict[str, Any]) -> List[int]:
157         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
158         (0) keep every window ? identical to the substrate. Persisted candidates are
159         untouched (full superset).
160
161         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
162         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
163         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
164         set rather than emit a video with zero rallies. This makes Gate A/B strictly
165         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
166         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
167         below it always wins on an otherwise-empty video. Observed on real footage
168         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
169         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
170         fcfg = idx_cfg.get("fusion", {})
171         min_crossings = int(fcfg.get("min_crossings", 0))
172         min_players = int(fcfg.get("min_players", 0))
173         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))

```

```

174
175         keep: List[int] = []
176         for i, st in enumerate(window_stats):
177             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
178                 continue
179             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
180                 continue
181                 keep.append(i)
182
183         if not keep and window_stats:
184             # Gating emptied a non-empty video ? fall back to the ungated set.
185             return list(range(len(window_stats)))
186         return keep
187
188
189     segmenter_registry.register(FusionSegmenter)
190

```

16. assistant (2026-06-20T08:57:45.346Z)

Now let me look at the new tests and how they exercise `cmd_train`, plus the stub runner's ``synth_trajectory_from_labels`` signature, and check whether ``runner.run_predict`` returns paths into ``traj_dir`` consistently with the harness manifest's expectations.

17. user (2026-06-20T08:57:46.270Z)

```

1     """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3     `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4     segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5     on the droplet against MinIO.
6     """
7     import json
8
9     import pytest
10
11     from backend.config import load_config
12     from backend.worker import __main__ as wmain
13
14
15     # ----- pure helpers
16
17     @pytest.mark.parametrize("raw,expected", [
18         ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid",
"wasb_hybrid"),
19     ])
20     def test_coerce(raw, expected):
21         assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)
22
23
24     def test_apply_overrides_on_typed_config():
25         cfg = _fresh_config()
26         wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
27                                     "sport=tennis"])
28         assert cfg.indexing.skip_ai_handoff is True
29         assert cfg.indexing.min_segment_duration == 1.5
30         assert cfg.sport == "tennis"
31
32
33     def test_apply_overrides_rejects_bad_format():
34         cfg = _fresh_config()
35         with pytest.raises(ValueError):

```

```

36         wmain._apply_overrides(cfg, ["no_equals_sign"])
37
38
39     def test_write_intervals_csv(tmp_path):
40         segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
41 "shot_count_confidence": "LocalNoAI"},
42                 {"start_time": 9.0, "end_time": 12.5}]
43         out = tmp_path / "intervals.csv"
44         wmain._write_intervals_csv(segs, str(out))
45         lines = out.read_text().splitlines()
46         assert lines[0] == "start,end,shot_count,ending_reason"
47         assert lines[1].startswith("2.000,5.000,4,")
48         assert lines[2].startswith("9.000,12.500,,")
49
50     def test_write_report_json(tmp_path):
51         out = tmp_path / "report.json"
52         wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success",
53 str(out))
54         data = json.loads(out.read_text())
55         assert data["video_id"] == "vid1" and data["segment_count"] == 1
56         assert data["segments"][0] == {"start": 1.0, "end": 2.0}
57
58     def test_parser_infer_accepts_set_and_uris():
59         args = wmain.build_parser().parse_args([
60             "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
61             "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
62         assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
63         assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
64
65
66     # ----- cmd_infer (local,
67 mocked)
68     class _OKResult:
69         passed = True
70         validator_name = "fake"
71         message = "ok"
72
73         def model_dump(self):
74             return {"validator_name": "fake", "passed": True, "message": "ok"}
75
76
77     class _FailResult(_OKResult):
78         passed = False
79         message = "too blurry"
80
81
82     class _FakeSeg:
83         def __init__(self, db, config):
84             pass
85
86         def process_video(self, video_id, path, max_frames=None):
87             return [
88                 {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
89 "shot_count_confidence": "LocalNoAI"},
90                 {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
91 "shot_count_confidence": "LocalNoAI"},
92             ]
93
94     def _fresh_config():
95         # Load defaults without touching any cwd config.json.
96         import tempfile

```

```

96     p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
97     p.write("{}")
98     p.close()
99     return load_config(p.name)
100
101
102 @pytest.fixture
103 def mocked_pipeline(monkeypatch):
104     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
105     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
106                         lambda path, cfg: ([_OKResult()], {}))
107     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
108     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
109
110
111 def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
112     video = tmp_path / "in.mp4"
113     video.write_bytes(b"FAKEVIDEO")
114     cfg = tmp_path / "config.json"
115     cfg.write_text("{}")
116     out = tmp_path / "bucket"
117     scratch = tmp_path / "scratch"
118
119     rc = wmain.main([
120         "infer", "--video-uri", str(video), "--out-uri", str(out),
121         "--config", str(cfg), "--scratch", str(scratch),
122         "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
123     ])
124     assert rc == 0
125     intervals = out / "outputs" / "vid123" / "intervals.csv"
126     report = out / "outputs" / "vid123" / "report.json"
127     assert intervals.exists() and report.exists()
128     rows = intervals.read_text().splitlines()
129     assert len(rows) == 3 # header + 2 rallies
130     assert json.loads(report.read_text())["segment_count"] == 2
131
132
133 def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
134     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
135     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
136                         lambda path, cfg: ([_FailResult()], {}))
137     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
138     video = tmp_path / "in.mp4"
139     video.write_bytes(b"X")
140     cfg = tmp_path / "config.json"
141     cfg.write_text("{}")
142     rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path /
143 "o"),
144                     "--config", str(cfg), "--scratch", str(tmp_path / "s"))])
145     assert rc == 2
146
147 # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
148 ---
149 def test_cmd_train_orchestration(tmp_path, monkeypatch):
150     import os
151     import subprocess
152
153     labels_dir = tmp_path / "corpus" / "labels"
154     labels_dir.mkdir(parents=True)
155     header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
156     (labels_dir / "v1.rallies.csv").write_text(header +
157 "1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
158     (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")

```

```

158 out_dir = tmp_path / "out"
159 scratch = tmp_path / "scratch"
160 cfg = tmp_path / "config.json"
161 cfg.write_text("{}")
162
163 # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
164 def fake_run(cmd, *a, **k):
165     out = cmd[cmd.index("--out") + 1]
166     ver = cmd[cmd.index("--version") + 1]
167     bdir = os.path.join(out, ver)
168     os.makedirs(bdir, exist_ok=True)
169     for fn in ("weights.json", "manifest.json"):
170         with open(os.path.join(bdir, fn), "w") as f:
171             f.write("{}")
172     return subprocess.CompletedProcess(cmd, 0)
173 monkeypatch.setattr(subprocess, "run", fake_run)
174
175 rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
176 str(out_dir),
177                 "--version", "0.0.1", "--config", str(cfg), "--scratch",
178 str(scratch)])
179 assert rc == 0
180 # bundle pushed to <out-uri>/weights/<version>/
181 assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
182 assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
183 # the manifest fed to the harness enumerated both labeled videos with a trajectory
184 each
185 specs = json.loads((scratch / "manifest.json").read_text())
186 assert len(specs) == 2
187 assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)
188
189 def test_parser_train_detector_args():
190     args = wmain.build_parser().parse_args([
191         "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version",
192         "1.0.0",
193         "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
194     assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
195
196 def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
197     """--trajectory-source detector pulls videos + runs the resolved DetectorRunner.
198 With
199 RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the
200 runner)."""
201     import os
202     import subprocess
203
204     monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub") # resolve a CPU stub runner (no
GPU/WSL)
205     corpus = tmp_path / "corpus"
206     labels_dir = corpus / "labels"
207     videos_dir = corpus / "videos"
208     labels_dir.mkdir(parents=True)
209     videos_dir.mkdir(parents=True)
210     header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
211     (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
212     (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
213     (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
214     (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
215     out_dir = tmp_path / "out"
216     scratch = tmp_path / "scratch"
217     cfg = tmp_path / "config.json"
218     cfg.write_text("{}")

```

```

216     def fake_run(cmd, *a, **k):
217         out = cmd[cmd.index("--out") + 1]
218         ver = cmd[cmd.index("--version") + 1]
219         bdir = os.path.join(out, ver)
220         os.makedirs(bdir, exist_ok=True)
221         for fn in ("weights.json", "manifest.json"):
222             with open(os.path.join(bdir, fn), "w") as f:
223                 f.write("{}")
224         return subprocess.CompletedProcess(cmd, 0)
225     monkeypatch.setattr(subprocess, "run", fake_run)
226
227     rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
228                     "--version", "0.0.2", "--config", str(cfg), "--scratch",
str(scratch),
229                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
230     assert rc == 0
231     specs = json.loads((scratch / "manifest.json").read_text())
232     assert len(specs) == 2
233     # Real-extract path: each spec points at an on-disk trajectory the runner actually
wrote.
234     assert all(os.path.exists(s["trajectory"]) for s in specs)
235     assert all(s["trajectory"].endswith("_stub.csv") for s in specs) # the stub
runner's output
236

```

18. user (2026-06-20T08:57:46.899Z)

```

1     """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3     A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4     CSV with **no GPU, no WSL, no model weights** ? so the whole trajectory-hybrid pipeline
5     (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6     container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7     (``use_gpu=False``); this closes the last GPU/WSL-only gap so the *entire* pipeline is
8     runnable ? and regression-gateable ? without a GPU.
9
10    Two modes:
11    * **replay** ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12      pipeline with a committed/golden trajectory; deterministic + cross-OS).
13    * **synth** (default) ? generate a deterministic synthetic trajectory of
14      ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
15      gaps, so windowing yields a stable, known number of candidate windows.
16
17    This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18    "Optional Linux-native path") and the **seed of the future ``LinuxNativeRunner``**
19    (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20    ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21    WSL runners stay the default, so production behaviour is unchanged.
22    """
23    import os
24    import csv as _csv
25    import shutil
26    import logging
27    from dataclasses import dataclass
28    from typing import Any, Dict, List, Optional, Tuple
29
30    from backend.pipeline.detectors.base import DetectorRunner
31    # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
32    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33
34    logger = logging.getLogger(__name__)
35
36
37    @dataclass
38    class StubConfig:

```

```

39         """Settings for the CPU stub runner (built from ``config.json`` ->
``indexing.stub``)."""
40         fps: float = 30.0
41         frame_width: float = 1280.0
42         num_rallies: int = 2
43         rally_seconds: float = 4.0
44         gap_seconds: float = 3.0          # dead gap between rallies (> merge_gap so
windows don't merge)
45         lead_seconds: float = 2.0        # dead lead-in
46         step_px: float = 12.0           # per-frame shuttle displacement during a rally
(>> velocity_thresh)
47         replay_csv: Optional[str] = None # if set + exists, emit this CSV verbatim instead
of synthesising
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
51         s = (idx_cfg or {}).get("stub", {}) or {}
52         return cls(
53             fps=float(s.get("fps", 30.0)),
54             frame_width=float(s.get("frame_width", 1280.0)),
55             num_rallies=int(s.get("num_rallies", 2)),
56             rally_seconds=float(s.get("rally_seconds", 4.0)),
57             gap_seconds=float(s.get("gap_seconds", 3.0)),
58             lead_seconds=float(s.get("lead_seconds", 2.0)),
59             step_px=float(s.get("step_px", 12.0)),
60             replay_csv=s.get("replay_csv"),
61         )
62
63
64     class StubDetectorRunner(DetectorRunner):
65         """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""
66
67         def __init__(self, cfg: Optional[StubConfig] = None):
68             self.cfg = cfg or StubConfig()
69
70         def healthcheck(self) -> Tuple[bool, str]:
71             return True, "stub CPU detector (no GPU/WSL)"
72
73         def _rows(self) -> List[Tuple[int, int, float, float]]:
74             """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight
bursts
75             separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
?
76             well above the default 0.02 velocity threshold ? so every rally frame is
"active"
77             and each burst becomes exactly one candidate window."""
78             cfg = self.cfg
79             rows: List[Tuple[int, int, float, float]] = []
80             frame = 0
81             y = cfg.frame_width / 4.0
82             base_x = cfg.frame_width / 2.0
83             for _ in range(int(cfg.lead_seconds * cfg.fps)):
84                 rows.append((frame, 0, 0.0, 0.0))
85                 frame += 1
86             for r in range(cfg.num_rallies):
87                 for i in range(int(cfg.rally_seconds * cfg.fps)):
88                     x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
89                     rows.append((frame, 1, x, y))
90                     frame += 1
91                 if r != cfg.num_rallies - 1:
92                     for _ in range(int(cfg.gap_seconds * cfg.fps)):
93                         rows.append((frame, 0, 0.0, 0.0))
94                         frame += 1
95             return rows
96

```



```

97         def run_predict(self, video_win_path: str, output_win_dir: str,
98                         video_id: Optional[str] = None) -> Optional[str]:
99             """Write a deterministic trajectory CSV and return its path (never touches the
GPU/WSL)."""
100             os.makedirs(output_win_dir, exist_ok=True)
101             stem = os.path.splitext(os.path.basename(str(video_win_path).replace("\\",
"/")))[0]
102             key = f"{stem}_{video_id[:12]}" if video_id else (stem or "stub")
103             out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")
104
105             if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):
106                 shutil.copyfile(self.cfg.replay_csv, out_csv)
107                 logger.info("Stub runner: replayed trajectory %s -> %s",
self.cfg.replay_csv, out_csv)
108             return out_csv
109
110             rows = self._rows()
111             with open(out_csv, "w", newline="") as f:
112                 w = _csv.writer(f)
113                 w.writerow(["Frame", "Visibility", "X", "Y"])
114                 for frame, vis, x, y in rows:
115                     w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
116             logger.info("Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows),
out_csv)
117             return out_csv
118
119             # Model-agnostic CSV parsing + windowing ? identical to the real runners.
120             parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
121             trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
122
123
124         def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
125                                       frame_width: float = 1280.0, tail_pad: float = 2.0,
126                                       step_px: float = 12.0) -> str:
127             """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
trajectory whose
128             in-flight (visible + oscillating) frames cover each labeled rally window and whose
dead frames
129             cover the gaps ? so the synthetic trajectory *encodes* the rally labels.
130
131             This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
data with
132             no GPU/WSL, validating the whole training pipeline before the real WASB detector
lands. (The
133             learned model is meaningful for plumbing, not for production ? the trajectory is
synthetic.)
134             """
135             rallies: List[Tuple[float, float]] = []
136             with open(labels_csv, newline="", encoding="utf-8") as f:
137                 for row in _csv.DictReader(f):
138                     try:
139                         rallies.append((float(row["start_time"]), float(row["end_time"])))
140                     except (KeyError, ValueError):
141                         continue
142             duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
143             nframes = max(1, int(duration * fps))
144             base_x = frame_width / 2.0
145             y = frame_width / 4.0
146             os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
147             with open(out_csv, "w", newline="", encoding="utf-8") as f:
148                 w = _csv.writer(f)
149                 w.writerow(["Frame", "Visibility", "X", "Y"])
150                 for i in range(nframes):
151                     t = i / fps

```

```

152         if any(s <= t < e for s, e in rallies):
153             x = base_x + (step_px if i % 2 else -step_px)
154             w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
155         else:
156             w.writerow([i, 0, "0.000", "0.000"])
157     return out_csv
158

```

19. assistant (2026-06-20T08:57:52.612Z)

I have a comprehensive understanding now. Let me check a couple more details: the `storage.parse\_uri` and `storage.list\_uris` behavior (to confirm `.name` and listing semantics), and whether the default synth path is byte-unchanged from before (the prior commit d5407eb). Let me check the storage module and the git diff of `\_\_main\_\_.py`.

20. user (2026-06-20T08:57:54.268Z)

```

diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index c7836ba..ccd362c 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -159,17 +159,49 @@ def cmd_infer(args: argparse.Namespace) -> int:
     return 0

+def _resolve_train_detector(args: argparse.Namespace, config: Any):
+    """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
+    detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU box,
+    stub=CPU mock). Returns the runner, or prints an error + returns None.
+
+    Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build the
+    detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo) has
+    no shuttle detector and is rejected.
+    """
+    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
+
+    seg_cls = segmenter_registry.get(args.segmenter)
+    if not seg_cls:
+        print(f"[worker] unknown segmenter: {args.segmenter!r} "
+              f"(have {list(segmenter_registry.list_available())})")
+        return None
+    seg = seg_cls(None, config)
+    if not isinstance(seg, TrajectoryHybridSegmenter):
+        print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
+              f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid).")
+        return None
+    runner, _ = seg._resolve_runner(config.get("indexing", {}))
+    ok, msg = runner.healthcheck()
+    if not ok:
+        print(f"[worker] detector environment not ready: {msg}")
+        return None
+    print(f"[worker] detector ready ({args.segmenter}): {msg}")
+    return runner
+
+def cmd_train(args: argparse.Namespace) -> int:
+    """Gen-0 train-from-bucket: pull labels -> synth CPU-mock trajectories -> manifest -> shell
+    out
+
+    """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest -> shell out
+    to training.gen0.harness (backend must NOT import training) -> push the artifact bundle.
+
+    The trajectories are the CPU "mock GPU": deterministic, label-consistent synthetic shuttle
+    paths
+    (real WASB extraction is the GPU/M3.5 path). This validates the whole train pipeline on
+    CPU.

```

```

+ Two trajectory sources (the train pipeline + harness are identical either way):
+ * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent synthetic
+ shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box / CI.
+ * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
+ DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test). This
+ is the M3.5 "first real training" path; only the trajectory source flips.
+ """
import subprocess

- from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
-
config = load_config(args.config) if args.config else load_config()
_apply_overrides(config, args.set)
storage_cfg = config.storage.model_dump()
@@ -177,6 +209,7 @@ def cmd_train(args: argparse.Namespace) -> int:
scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
labels_dir = os.path.join(scratch, "labels")
traj_dir = os.path.join(scratch, "trajectories")
+ videos_dir = os.path.join(scratch, "videos")
os.makedirs(labels_dir, exist_ok=True)
os.makedirs(traj_dir, exist_ok=True)

@@ -188,19 +221,56 @@ def cmd_train(args: argparse.Namespace) -> int:
    f"under {corpus}/labels/")
    return 2

+ # Real-detector source: resolve the runner once + index the bucket's videos/ by stem.
+ runner = None
+ video_by_stem: dict = {}
+ if args.trajectory_source == "detector":
+     os.makedirs(videos_dir, exist_ok=True)
+     runner = _resolve_train_detector(args, config)
+     if runner is None:
+         return 2
+     for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
+         vname = storage.parse_uri(vuri).name
+         video_by_stem[os.path.splitext(vname)[0]] = vuri
+
+ print(f"[worker] trajectory source: {args.trajectory_source}")
+ specs: List[dict] = []
+ for uri in label_uris:
+     fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
+     name = fname.replace(".rallies.csv", "").replace(".csv", "")
+     local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
+     traj = os.path.join(traj_dir, f"{name}_traj.csv")
+     synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
+     specs.append({"name": name, "trajectory": traj, "labels": local_label,
+         "fps": args.fps, "frame_width": args.frame_width})
+     if args.trajectory_source == "detector":
+         vuri = video_by_stem.get(name)
+         if not vuri:
+             print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
+             continue
+         local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
+             config=storage_cfg)
+         video_id = compute_video_id(local_video)
+         # Real fps/width drive the trajectory frame->time mapping (synth fixes them at
30/1280).
+         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
+         fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
+         traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
+         if not traj:

```

```

+         print(f"[worker] detector produced no trajectory for {name!r} ? skipping.")
+         continue
+         specs.append({"name": name, "trajectory": traj, "labels": local_label,
+                       "fps": fps, "frame_width": frame_width})
+     else:
+         from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
+         traj = os.path.join(traj_dir, f"{name}_traj.csv")
+         synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
+         specs.append({"name": name, "trajectory": traj, "labels": local_label,
+                       "fps": args.fps, "frame_width": args.frame_width})
+
+     if len(specs) < 2:
+         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
+         return 2
+         manifest_path = os.path.join(scratch, "manifest.json")
+         with open(manifest_path, "w", encoding="utf-8") as f:
+             json.dump(specs, f, indent=2)
-         print(f"[worker] built manifest: {len(specs)} videos (CPU-mock stub trajectories)")
+         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
+         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

+         out_dir = os.path.join(scratch, "artifacts")
+         cmd = [sys.executable, "-m", "training.gen0.harness",
@@ -245,15 +315,23 @@ def build_parser() -> argparse.ArgumentParser:
+             help="config override, e.g. --set indexing.skip_ai_handoff=true")
+         pi.set_defaults(func=cmd_infer)

-         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels -> stub trajectories ->
harness)")
-         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (e.g.
s3://bucket)")
+         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
+         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+ videos/
for "
+
+             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
+         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/ is
appended)")
+         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
+         pt.add_argument("--trajectory-source", choices=["synth", "detector"], default="synth",
+             help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
+             "detector = REAL shuttle trajectories via the resolved DetectorRunner
"
+
+             "(needs videos/ in the corpus; honours detector_impl).")
+         pt.add_argument("--segmenter", default="wasb_hybrid",
+             help="trajectory hybrid whose detector is used for --trajectory-source
detector")
+         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or storage
URI)")
+         pt.add_argument("--config", default=None)
+         pt.add_argument("--scratch", default=None)
-         pt.add_argument("--fps", type=float, default=30.0)
-         pt.add_argument("--frame-width", type=float, default=1280.0)
+         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps (detector
probes the video)")
+         pt.add_argument("--frame-width", type=float, default=1280.0,
+             help="synth trajectory frame width (detector probes the video)")
+         pt.add_argument("--set", action="append", default=[], metavar="k=v")
+         pt.set_defaults(func=cmd_train)
+         return p

```

```

"""Pluggable storage layer: local (default) + S3/GCS first-class buckets + Drive (follow-up).

URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are imported
lazily, only when a non-local URI is actually used (install the ``storage-s3`` / ``storage-gcs``
/ ``storage-gdrive`` extras). The default ``local`` backend needs no third-party deps and
preserves today's behaviour byte-for-byte.

Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an identity
passthrough. See ``docs/DECOUPLED_COMPUTE/``.
"""

from __future__ import annotations

import os
from typing import Any, Dict, Iterator, Optional

from backend.storage.base import StorageBackend
from backend.storage.ref import StorageRef, StorageStat, parse_uri

__all__ = [
    "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
    "get_storage", "fetch", "put", "list_uris",
]

def _backend_class(name: str):
    """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
    if name == "local":
        from backend.storage.local import LocalStorage
        return LocalStorage
    if name == "s3":
        from backend.storage.s3 import S3Storage
        return S3Storage
    if name == "gcs":
        from backend.storage.gcs import GCSStorage
        return GCSStorage
    if name == "gdrive":
        from backend.storage.gdrive import GDriveStorage
        return GDriveStorage
    raise ValueError(f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive)")

def get_storage(backend: Optional[str] = None, *,
                config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
    """Construct a backend. ``backend`` (or ``config['backend']``, default 'local') selects
    the implementation; its per-backend settings come from ``config[backend]`` merged with
    ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-selectors."""
    cfg = config or {}
    name = backend or cfg.get("backend", "local")
    settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
    settings.update(overrides)
    return _backend_class(name)(**settings)

def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
          config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
    """Resolve ``uri`` to a local file path. For ``local`` this is an identity passthrough
    (no copy); for s3/gcs it downloads to ``dst``."""
    ref = parse_uri(uri)
    return get_storage(ref.backend, config=config).fetch_to_local(ref, dst, overwrite=overwrite)

def put(local_path: "str | os.PathLike[str]", uri: str, *,
        config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) -> str:
    """Upload/copy ``local_path`` to ``uri``; return ``uri``."""

```

```

    ref = parse_uri(uri)
    get_storage(ref.backend, config=config).put(local_path, ref, content_type=content_type)
    return uri

def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) -> Iterator[str]:
    """Yield the URI of every object under ``prefix_uri``."""
    ref = parse_uri(prefix_uri)
    for r in get_storage(ref.backend, config=config).list(ref):
        yield r.uri

```

22. assistant (2026-06-20T08:58:01.152Z)

Now I have everything I need to assess the synth path (byte-unchanged) and analyze the detector path. Let me verify a few important subtleties:

1. The synth path: previously `len(specs) < 2` was NOT checked post-loop (the old code only checked `label_uris < 2` pre-loop). Now there's a new `if len(specs) < 2` post-loop check. For synth, every label produces a spec (no skip), so post-loop count == pre-loop count, so behavior is unchanged. Good ? that's byte-safe.

2. Let me check `parse_uri`/`list_uris`` ordering for the detector videos ? is `video_by_stem`` deterministic, and could a video listing collision overwrite? And critically: does `list_uris`` for `videos/`` recurse and potentially pick up non-video files?

Let me check the LocalStorage `list`` and `parse_uri``.

23. user (2026-06-20T08:58:02.034Z)

```

1      """Local-filesystem storage backend ? the default, zero new dependencies.
2
3      Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
4      file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
5      the whole pipeline behaves exactly as before the storage layer existed.
6      """
7      from __future__ import annotations
8
9      import os
10     import shutil
11     from datetime import datetime
12     from typing import Iterator
13
14     from backend.storage.base import StorageBackend
15     from backend.storage.ref import StorageRef, StorageStat
16
17
18     class LocalStorage(StorageBackend):
19         def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20             # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21             self.base_dir = base_dir
22
23         def _path(self, ref: StorageRef) -> str:
24             p = ref.key
25             if self.base_dir and not os.path.isabs(p):
26                 return os.path.join(self.base_dir, p)
27             return p
28
29         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30             # The object is already on the local FS ? return its path (identity). ``dst`` is
31             # honoured only if it names a *different* location (a copy is then made).
32             src = self._path(ref)
33             if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                 return src

```

```

35         dst = str(dst)
36         if os.path.exists(dst) and not overwrite:
37             return dst
38         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39         shutil.copyfile(src, dst)
40         return dst
41
42     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43         dst = self._path(ref)
44         if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46             shutil.copyfile(str(local_path), dst)
47         return ref
48
49     def exists(self, ref) -> bool:
50         return os.path.exists(self._path(ref))
51
52     def stat(self, ref) -> StorageStat:
53         st = os.stat(self._path(ref))
54         return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56     def list(self, prefix) -> Iterator[StorageRef]:
57         base = self._path(prefix)
58         if os.path.isdir(base):
59             for root, _dirs, files in os.walk(base):
60                 for fn in sorted(files):
61                     full = os.path.join(root, fn)
62                     yield StorageRef("local", "", full, f"local://{full}")
63         elif os.path.exists(base):
64             yield StorageRef("local", "", base, f"local://{base}")
65
66     def delete(self, ref) -> None:
67         path = self._path(ref)
68         if os.path.exists(path):
69             os.remove(path)
70

```

24. user (2026-06-20T08:58:02.614Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5         <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6         local://<path>              -> local    (local:///abs or local://rel/p)
7         s3://<bucket>/<key>         -> s3        (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8         gcs://<bucket>/<key>       -> gcs        (gs:// accepted, normalized to gcs://)
9         gdrive://<folderId>/<path> -> gdrive    (follow-up backend)
10
11     No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12     stays cheap.
13     """
14     from __future__ import annotations
15
16     import re
17     from dataclasses import dataclass
18     from datetime import datetime
19     from typing import Optional
20
21     _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22     _KNOWN = ("local", "s3", "gcs", "gdrive")

```

```

23
24
25 @dataclass(frozen=True)
26 class StorageStat:
27     """Lightweight object metadata (uniform across backends)."""
28     size: int
29     modified: Optional[datetime] = None
30     etag: Optional[str] = None
31     content_type: Optional[str] = None
32
33
34 @dataclass(frozen=True)
35 class StorageRef:
36     """A backend + a location. ``bucket`` is the S3/GCS bucket or the Drive folder-id;
37     for ``local`` it is empty and ``key`` carries the filesystem path."""
38     backend: str
39     bucket: str
40     key: str
41     uri: str
42
43     @property
44     def name(self) -> str:
45         return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48 def parse_uri(uri: str) -> StorageRef:
49     """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51     A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52     have no ``//``) is treated as a local filesystem path ? preserving today's
53     behaviour.
54
55     """
56     s = str(uri)
57     m = _SCHEME_RE.match(s)
58     if not m:
59         return StorageRef("local", "", s, f"local://{s}")
60     scheme = m.group(1).lower()
61     rest = s[m.end():]
62     if scheme == "gs":
63         scheme = "gcs"
64     if scheme == "local":
65         return StorageRef("local", "", rest, s)
66     if scheme in _KNOWN:
67         bucket, _, key = rest.partition("/")
68         return StorageRef(scheme, bucket, key, s)
69     raise ValueError(f"unsupported storage URI scheme: {scheme}://{rest} (in {s})")

```

25. assistant (2026-06-20T08:58:14.284Z)

Now I have a full picture. Let me verify the key bug hypotheses with concrete reasoning by examining critical edge cases.

Critical observations:

1. **``video\_by\_stem`` only indexes ``videos/`` non-recursively-safe:**
`list_uris(f"{corpus}/videos/")` uses `os.walk` which recurses, picking up ALL files (including any non-video sidecar files), and `video_by_stem[stem] = vuri` overwrites on stem collision (e.g. `v1.mp4` and `v1.mov`, or `v1.mp4` in a subdir). Last-listed wins.
2. **The `run_predict(local_video, traj_dir, ...)` writes into the SAME `traj_dir` for both synth and detector**, but the detector CSV filename is `{stem}__{vid12}_wasb.csv` (or `_stub.csv`), while synth writes `{name}_traj.csv`. No collision between sources since they don't mix. But within detector source, two videos with the **same stem but different content** would get different `vid12` (MD5), so no collision. Good. But two labels mapping to the same

video stem ? handled by dict, only one survives in `video\_by\_stem`.

3. `**_probe_fps_width` on a fake/corrupt video`**`: `cv2.VideoCapture` won't open ? returns fallback `(30.0, 1280.0)`. That's silent ? but acceptable as fallback. However the real concern: the detector runner then runs on that same unreadable video. With the stub it succeeds anyway (ignores video content). With native WASB it'd fail in `run_predict` ? returns None ? skip. OK.

Let me check one more thing ? the `_probe_fps_width` is called BEFORE `run_predict`, and `cv2.VideoCapture` on a non-video file. Let me also confirm the harness manifest's expected keys to ensure `fps`/`frame_width` from detector path match what harness consumes.

26. user (2026-06-20T08:58:15.981Z)

```
3: python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
7:- TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
12: <out>/<version>/manifest.json (the reviewable contract: feature-schema version,
40: labels_for, run as lovo_run, _seg_f1,
45:# Post-processing constants ? part of the model's identity, recorded in the manifest.
```

27. user (2026-06-20T08:58:17.136Z)

```
7:- TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
40: labels_for, run as lovo_run, _seg_f1,
84: pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
85: arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
86: X, times = features(arr, float(s["fps"]))
87: gts = load_gt_intervals(s["labels"], strict=False)
89: "y": labels_for(times, gts), "gts": gts})
179: "videos": [{ "name": s["name"], "labels": s["labels"],
180:              "gt_hash": gt_hash(load_gt_intervals(s["labels"], strict=False)),
181:              "fps": s["fps"]} for s in specs],
192: "no_paid_llm_training_labels": True,
214: help="corpus manifest JSON: [{name,trajectory,fps,frame_width,labels},
...]"
```

28. assistant (2026-06-20T08:58:32.122Z)

The harness consumes `s["trajectory"]`, `s["frame_width"]`, `s["fps"]`, `s["labels"]`, `s["name"]`. The detector path provides all of these. Good ? manifest contract is satisfied.

Now one critical correctness concern in the `**detector path**`: `s["frame_width"]` comes from `_probe_fps_width` which returns `cap.get(cv2.CAP_PROP_FRAME_WIDTH)` ? the `**pixel width of the video frame**`. But the WASB/TrackNet trajectory CSV `X` coordinates ? are they in `**pixels**` or normalized? `build_frame_arrays(pts, frame_width, fps)` normalizes by `frame_width`. The WASB runner emits pixel coordinates, so dividing by pixel width is correct. The synth path uses `frame_width=1280` and emits coords around `1280/2`. Consistent. Good.

Now let me verify the `**key concern from the prompt**`: does `seg_cls(None, config)` then `_resolve_runner(config.get("indexing", {}))` work? `config` is `AppConfig`. `seg_cls(None, config)` sets `self.config = config` (the `AppConfig` model). The `_resolve_runner` is called with `config.get("indexing", {})` ? returns the indexing section as a `**dict**` (via `model_dump`). Inside `_resolve_runner`, `idx_cfg.get(...)` works on dict. And `NativeWasbConfig.from_indexing_cfg(idx_cfg)` does `idx_cfg.get("wasb", {})` ? dict. Good. The `FusionSegmenter.__init__` also references `self.config` but only in `_person_features` (not reached here). So construction with `AppConfig` is fine.

Let me double-check the `healthcheck` is being called `**twice**` for the detector path (once in `_resolve_train_detector` and once... no, `run_predict` doesn't healthcheck; only the segmenter's `process_video` does). So in `cmd_train` detector path, `healthcheck` runs once in `_resolve_train_detector`. Good ? but note that `process_video` is NOT called here (worker builds the manifest directly), so the per-window thresholds/persistence are skipped, which is intended.

Now let me check a subtle bug: in `_resolve_train_detector`, for `FusionSegmenter`,

`\_resolve\_runner` returns `(runner, params)` but the runner from `\_build\_runner` (auto path) for fusion with `tracknet` substrate would build a TrackNetRunner. That's fine. But the ****healthcheck for the stub**** always returns True regardless of env ? that's the intent for the wiring test.

Let me verify one real bug: ****`\_probe\_fps\_width`** opens the video with cv2 but the order of `cap.get` calls when **not** `cap.isOpened()`. Let me also confirm whether there's a concern when `cv2` is unavailable. It's imported at module top in trajectory_hybrid, so importing it in the worker via that path is fine.

Let me check the test for native runner to confirm parity expectations and look for the `wasb_infer.py`` device threading diff quickly.

29. user (2026-06-20T08:58:32.775Z)

```
1      """NativeWasbRunner coverage ? the Linux-native (no-WSL) WASB detector path (M3.5).
2
3      The native subprocess is mocked (no torch/conda/GPU here). These tests lock the contract
4      the GPU-box byte-parity gate then relies on: it IS a DetectorRunner, it is NOT a WSL
runner
5      (no WslCommandMixin / wsl invocation), it builds a native argv (no `wsl`, no `conda
6      activate`, no `/mnt` translation) that runs the SAME wasb_infer.py with the SAME output
CSV
7      name as the WSL runner, threads the device through, and cleans the frame cache on
success.
8      """
9      import os
10     import types
11     from unittest.mock import patch
12
13     import pytest
14
15     from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
16     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin
17
18
19     def _proc(rc=0, stdout="", stderr=""):
20         return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)
21
22
23     # --- identity / no-WSL guarantee
-----
24     def test_is_a_detector_runner():
25         assert isinstance(NativeWasbRunner(NativeWasbConfig()), DetectorRunner)
26
27
28     def test_is_not_a_wsl_runner():
29         """The whole point of M3.5: the native runner must carry NO WSL plumbing."""
30         r = NativeWasbRunner(NativeWasbConfig())
31         assert not isinstance(r, WslCommandMixin)
32         assert not hasattr(r, "_wsl_bash")
33
34
35     def test_reuses_model_agnostic_windowing():
36         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37         assert NativeWasbRunner.parse_trajectory_csv is TrackNetRunner.parse_trajectory_csv
38         assert NativeWasbRunner.trajectory_to_action_windows is
TrackNetRunner.trajectory_to_action_windows
39
40
41     # --- config resolution (env overrides win over config over default)
-----
42     def test_from_indexing_cfg_defaults():
43         cfg = NativeWasbConfig.from_indexing_cfg({})
```

```

44     assert cfg.device == "cuda" and cfg.python_bin == "python"
45     assert cfg.weights_path == "" and cfg.repo_dir == "~/models/WASB-SBDT"
46     assert cfg.stream_video is False and cfg.keep_frames is False
47
48
49     def test_from_indexing_cfg_reads_config_block():
50         cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
51             "weights_path": "/m/w.pth.tar", "device": "cpu", "python_bin":
"/envs/wasb/bin/python",
52             "repo_dir": "/m/WASB-SBDT", "stream_video": True, "keep_frames": True, "sport":
"tennis"}})
53         assert cfg.weights_path == "/m/w.pth.tar" and cfg.device == "cpu"
54         assert cfg.python_bin == "/envs/wasb/bin/python" and cfg.repo_dir == "/m/WASB-SBDT"
55         assert cfg.stream_video is True and cfg.keep_frames is True and cfg.sport ==
"tennis"
56
57
58     def test_env_overrides_beat_config(monkeypatch):
59         monkeypatch.setenv("WASB_WEIGHTS", "/env/w.pth.tar")
60         monkeypatch.setenv("WASB_DEVICE", "mps")
61         monkeypatch.setenv("WASB_PYTHON", "/env/py")
62         monkeypatch.setenv("WASB_REPO_DIR", "/env/repo")
63         monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
64         cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
65             "weights_path": "/cfg/w", "device": "cpu", "python_bin": "/cfg/py", "repo_dir":
"/cfg/repo"}})
66         assert cfg.weights_path == "/env/w.pth.tar" and cfg.device == "mps"
67         assert cfg.python_bin == "/env/py" and cfg.repo_dir == "/env/repo"
68         assert cfg.scratch_dir == "/env/scratch"
69
70
71     # --- healthcheck
-----
72     def _ok_cfg(tmp_path, **kw):
73         w = tmp_path / "w.pth.tar"
74         w.write_text("weights")
75         (tmp_path / "repo" / "src").mkdir(parents=True)
76         return NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "repo"), **kw)
77
78
79     def test_healthcheck_ok(tmp_path):
80         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
81         with patch.object(r, "_run", return_value=proc(0, stdout="torch 1.11.0 cuda
True")):
82             ok, msg = r.healthcheck()
83             assert ok and "torch" in msg
84
85
86     def test_healthcheck_empty_weights():
87         ok, msg = NativeWasbRunner(NativeWasbConfig(weights_path="")).healthcheck()
88         assert not ok and "weights" in msg.lower()
89
90
91     def test_healthcheck_weights_missing(tmp_path):
92         (tmp_path / "repo" / "src").mkdir(parents=True)
93         cfg = NativeWasbConfig(weights_path=str(tmp_path / "nope.pth"),
repo_dir=str(tmp_path / "repo"))
94         ok, msg = NativeWasbRunner(cfg).healthcheck()
95         assert not ok and "not found" in msg.lower()
96
97
98     def test_healthcheck_repo_missing(tmp_path):
99         w = tmp_path / "w.pth.tar"
100         w.write_text("x")
101         cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))

```

```

102         ok, msg = NativeWasbRunner(cfg).healthcheck()
103         assert not ok and "repo" in msg.lower()
104
105
106     def test_healthcheck_torch_import_fails(tmp_path):
107         r = NativeWasbRunner(_ok_cfg(tmp_path))
108         with patch.object(r, "_run", return_value=_proc(1, stderr="ModuleNotFoundError:
torch")):
109             ok, msg = r.healthcheck()
110             assert not ok and "failed" in msg.lower()
111
112
113     def test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
114         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
115         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
116             ok, msg = r.healthcheck()
117             assert not ok and "cuda" in msg.lower()
118
119
120     def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
121         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
122         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
123             ok, msg = r.healthcheck()
124             assert ok # cpu run does not require a GPU
125
126
127     def test_healthcheck_python_not_found(tmp_path):
128         r = NativeWasbRunner(_ok_cfg(tmp_path, python_bin="/no/such/python"))
129         with patch.object(r, "_run", side_effect=FileNotFoundError()):
130             ok, msg = r.healthcheck()
131             assert not ok and "python" in msg.lower()
132
133
134     # --- run_predict
135     -----
136     def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
137         """Drive run_predict to success with the subprocess mocked; returns (runner, out,
rm_spy, argv, cwd)."""
138         r = NativeWasbRunner(cfg)
139         key = f"clip__{vid[:12]}"
140         (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n") # success = CSV
exists
141         with patch.object(r, "_copy_infer_script", return_value=True), \
142             patch.object(r, "_run", return_value=_proc(0)) as run_spy, \
143             patch.object(r, "_rm_rf") as rm_spy:
144             out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
145             argv = run_spy.call_args[0][0]
146             cwd = run_spy.call_args.kwargs.get("cwd")
147             return r, out, rm_spy, argv, cwd
148
149     def test_run_predict_builds_native_argv_no_wsl(tmp_path):
150         _, out, rm, argv, cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
151         assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
152         # No WSL / conda plumbing anywhere in the command.
153         assert "wsl" not in argv
154         assert not any("conda" in a or "/mnt/" in a for a in argv)
155         # It runs the SAME inference core, device-threaded, writing the SAME CSV name as
WSL.
156         assert argv[0] == "python" and argv[1] == "wasb_infer.py"
157         assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
158         assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"

```

```

159     assert argv[argv.index("--out") + 1] == out
160     # Disk mode extracts frames + cleans them on success.
161     assert "--frames_out_dir" in argv and "--stream-video" not in argv
162     assert cwd.endswith(os.path.join("src"))
163     rm.assert_called_once()
164
165
166     def test_run_predict_stream_mode_no_frames_dir(tmp_path):
167         _, out, rm, argv, _cwd, _key = _drive_success(
168             NativeWasbConfig(weights_path="/m/w.pth", stream_video=True), tmp_path)
169         assert out is not None
170         assert "--stream-video" in argv and "--frames_out_dir" not in argv
171         rm.assert_not_called() # streaming never materializes a frame cache
172
173
174     def test_run_predict_device_threaded_cpu(tmp_path):
175         _, _out, _rm, argv, _cwd, _key = _drive_success(
176             NativeWasbConfig(weights_path="/m/w.pth", device="cpu"), tmp_path)
177         assert argv[argv.index("--device") + 1] == "cpu"
178
179
180     def test_run_predict_keep_frames_skips_cleanup(tmp_path):
181         _, _out, rm, _argv, _cwd, _key = _drive_success(
182             NativeWasbConfig(weights_path="/m/w.pth", keep_frames=True), tmp_path)
183         rm.assert_not_called()
184
185
186     def test_run_predict_none_when_copy_infer_fails(tmp_path):
187         r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
188         with patch.object(r, "_copy_infer_script", return_value=False):
189             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
190 is None
191
192     def test_run_predict_none_when_subprocess_fails(tmp_path):
193         r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
194         with patch.object(r, "_copy_infer_script", return_value=True), \
195             patch.object(r, "_run", return_value=_proc(1, stderr="CUDA error")):
196             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
197 is None
198
199     def test_run_predict_none_when_csv_absent(tmp_path):
200         # Subprocess "succeeds" (rc 0) but writes no CSV -> treated as failure.
201         r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
202         with patch.object(r, "_copy_infer_script", return_value=True), \
203             patch.object(r, "_run", return_value=_proc(0)):
204             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
205 is None
206
207     def test_run_predict_times_out_returns_none(tmp_path):
208         import subprocess
209         r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
210         with patch.object(r, "_copy_infer_script", return_value=True), \
211             patch.object(r, "_run", side_effect=subprocess.TimeoutExpired(cmd="x",
212 timeout=1)):
213             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
214 is None
215
216     def test_csv_name_matches_wsl_contract(tmp_path):
217         """The native CSV name must equal the WSL runner's so downstream + the parity diff
line up."""
218         _, out, _rm, _argv, _cwd, key =

```

```

_drive_success(NativeWasbConfig(weights_path="/m/w.pth"), tmp_path)
218         assert os.path.basename(out) == f"{key}_wasb.csv"
219
220
221     # --- the detector_impl='native' seam (parallels the stub seam)
-----
222     def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
223         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
224         from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
225         runner, params = WasbHybridSegmenter(None, {}).resolve_runner({"detector_impl":
"native"})
226         assert isinstance(runner, NativeWasbRunner)
227         assert params.get("detector_impl") == "native"
228
229
230     def test_resolve_runner_native_env_overrides_config(monkeypatch):
231         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
232         from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
233         runner, _ = WasbHybridSegmenter(None, {}).resolve_runner({"detector_impl": "auto"})
234         assert isinstance(runner, NativeWasbRunner)
235
236
237     def test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
238         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
239         from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
240         runner, params = FusionSegmenter(None, {}).resolve_runner({"detector_impl":
"native"})
241         assert isinstance(runner, NativeWasbRunner)
242         assert params.get("fusion_substrate") == "wasb"
243
244
245     def test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
246         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
247         from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
248         with pytest.raises(NotImplementedError):
249             FusionSegmenter(None, {}).resolve_runner(
250                 {"detector_impl": "native", "fusion": {"substrate": "tracknet"}})
251
252
253     def test_resolve_runner_native_refused_for_tracknet(monkeypatch):
254         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
255         from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
256         with pytest.raises(NotImplementedError):
257             TrackNetHybridSegmenter(None, {}).resolve_runner({"detector_impl": "native"})
258

```

30. assistant (2026-06-20T08:58:35.873Z)

Now I have full context. Let me run the worker tests + native runner tests to confirm the current state is green and to validate my reasoning about the `config.get` path actually works at runtime.

31. user (2026-06-20T08:58:45.666Z)

```

..... [100%]
43 passed in 6.58s

```

32. assistant (2026-06-20T08:58:53.472Z)

Tests pass. Now let me verify the key potential bug: the `\_resolve\_train\_detector` healthcheck for the `detector` path with a NON-stub runner. Note that `seg.resolve\_runner` for the `auto`/`native` path doesn't catch exceptions from `\_build\_native\_runner` (e.g. fusion+tracknet raises `NotImplementedError`). In `\_resolve\_train\_detector`, that would propagate as an uncaught exception rather than a clean error+return None. Let me confirm by checking ? `\_resolve\_runner`

calls `self._build_native_runner(idx_cfg)` which can raise `NotImplementedError` for `tracknet_hybrid` or `fusion+tracknet` substrate. `_resolve_train_detector` does NOT wrap this in `try/except`, so `cmd_train` would crash with a traceback instead of returning exit code 2.

Let me also verify the `**fps probe bug**`: `_probe_fps_width` calls `cap.get()` twice with two separate `isOpened()` checks. If a video opens but reports fps but the `cap` gets released between... no, it's sequential on the same cap. Fine.

Let me now write a quick test to confirm the `NotImplementedError` propagation bug is real (`tracknet_hybrid` via detector source).

33. user (2026-06-20T08:59:03.058Z)

RAISED: `NotImplementedError` detector_impl='native' is not supported for the 'TrackNet' family ? only WASB has a Linux-native runner. Use detector_im

34. assistant (2026-06-20T08:59:14.070Z)

Confirmed bug: `_resolve_train_detector` lets `NotImplementedError` (and any `_build*_runner` exception) escape uncaught, so `cmd_train` crashes with a traceback instead of returning a clean exit code 2. This is a real misconfiguration path: `--segmenter tracknet_hybrid` (or `fusion_hybrid` with `tracknet` substrate) + `detector_impl=native`.

Now let me check another path ? what happens with `--segmenter wasb_hybrid` + the default `auto` (WSL) impl on the Linux GPU box. That builds a `WasbRunner` (WSL). On Linux there's no `wsl` binary, so `healthcheck()` would fail cleanly (returns False) ? handled. Good.

Let me verify one more real concern: the `**synth path post-loop len(specs) < 2` check changes the error code semantics. Previously (origin/master), was there a post-loop guard? From the diff, the OLD synth code had NO `if len(specs) < 2` post-loop ? it went straight to writing the manifest. Now there IS one. For synth, specs always equals `label_uris` count (no skips), and pre-loop already guarantees `>= 2`, so the new check never triggers for synth ? byte-identical behavior. Confirmed safe.

Now let me check the `**video/label stem matching collision**` concretely and the `**non-recursive listing**` issue. `list_uris(videos/)` walks recursively and indexes every file by stem. Consider `videos/v1.mp4` and `videos/v1.srt` (subtitle) ? both have stem `v1`, dict overwrite ? the `.srt` could win (sorted order: `v1.mp4` < `v1.srt`, walk yields sorted within dir, so `.srt` is listed last ? overwrites `.mp4`). Then `compute_video_id` on a subtitle, runner runs on it, fails ? skip. Not catastrophic but a real footgun. Let me verify sorted order and the overwrite.

35. user (2026-06-20T08:59:16.555Z)

```
sorted: ['v1.json', 'v1.mp4', 'v1.srt']
video_by_stem[v1] = v1.srt
```

36. assistant (2026-06-20T08:59:28.936Z)

Confirmed: `video_by_stem` indexes ALL files under `videos/` recursively (including sidecars like `.srt/.json/.txt`), and a stem collision makes the last-sorted file win ? so `v1.srt` shadows `v1.mp4`. The runner then runs on a non-video file ? fails or produces garbage. The matching should filter to video extensions (mirroring how labels are filtered to `.csv`).

Now let me check the `list_uris(labels/)` filter ? it filters `.csv` but ALSO recursively. If there's a nested `labels/sub/x.csv` it'd be picked up. That's pre-existing though (not in this diff for labels). The video side is new and unfiltered.

Let me verify one more important correctness issue: when `--trajectory-source detector` and the runner is the `**stub**`, `run_predict` writes to `traj_dir` with name `{stem}__{vid12}_stub.csv`. Two videos `v1.mp4` and `v2.mp4` ? distinct stems ? distinct files. Good. But what if `videos/` has `v1.mp4` and a label `v1.rallies.csv` plus `v1` also matches ? fine.

Now, a subtle one: `**name` derivation from labels vs `stem` from videos. Label `Gx020094_proxy.rallies.csv` ? `name = "Gx020094_proxy"`. Video must be `Gx020094_proxy.mp4` ?

stem `GX020094\_proxy`. Match works. But if the video is `GX020094\_proxy.rallies.mp4` (unlikely) the stem would be `GX020094\_proxy.rallies` ? name. Edge case, acceptable.

Let me check whether there's a concern that `videos\_dir` fetch destination collides when two video URIs share a basename (different subdirs). `storage.fetch(vuri, os.path.join(videos\_dir, parse\_uri(vuri).name))` ? if `videos/a/v.mp4` and `videos/b/v.mp4` both have name `v.mp4`, they'd fetch to the same dest `videos\_dir/v.mp4`. But `video\_by\_stem` already collapsed them to one entry (last wins), so only one is ever fetched. Still ? the recursion is the root issue.

Let me run ruff/mypy on the changed files to surface any lint/type issues for the review gate.

37. user (2026-06-20T08:59:31.109Z)

All checks passed!

38. user (2026-06-20T08:59:34.559Z)

```
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 2 source files
```

39. assistant (2026-06-20T08:59:46.870Z)

Ruff and mypy clean. Now let me consider the `**_probe_fps_width` when cv2 can't open** more carefully for the detector path, and confirm there's no double-fetch / wasted work, plus check whether a `--trajectory-source detector` run with NO `videos/` dir at all gives a clean message.

```
`list_uris(f"{corpus}/videos/")` when `videos/` doesn't exist: `LocalStorage.list` checks
`os.path.isdir(base)` ? False, `os.path.exists(base)` ? False ? yields nothing. So
`video_by_stem` is empty ? every label hits "no video for ... skipping" ? `len(specs) < 2` ?
returns 2 with a reasonable message. OK, clean.
```

Now, one more real correctness concern: `**_probe_fps_width` runs BEFORE `run_predict`, and it opens the video via cv2 even on the GPU box where the runner itself decodes.** That's a minor double-open, not a bug. But more importantly: for the `**detector path` the fps probe failure is silent ? if cv2 returns 0 (e.g. an unusual codec cv2 can't read but ffmpeg/WASB can), it falls back to 30/1280 and the trajectory?time mapping in the harness would be WRONG (frame indices divided by 30 when the real video is 60fps ? all rally times halved). That's a silent data-corruption risk in the training manifest. Worth flagging as major because it directly corrupts the `fps/frame_width` that the harness uses for `build_frame_arrays` and time mapping. Let me confirm WASB CSV is frame-indexed (so fps matters for time).

The harness line 85-86: `build_frame_arrays(pts, frame_width, fps)` then `features(arr, fps)`. So yes, fps is load-bearing for frame?time. A wrong probe silently corrupts labels alignment. The synth path doesn't have this issue (it generates the trajectory at the SAME fps it records). But the detector path probes a real video and a probe miss is silent.

Let me check `get_video_duration` and whether there's a shared, more-robust ffprobe-based helper that should be used instead of cv2 for consistency.

40. user (2026-06-20T08:59:47.900Z)

```
7: def get_video_duration(video_path: str) -> float:
8:     """Returns the exact duration of a video file in seconds using ffprobe.
12:         "ffprobe", "-v", "error", "-show_entries", "format=duration",
```

41. assistant (2026-06-20T08:59:56.676Z)

So `get_video_duration` uses ffprobe (robust), but `_probe_fps_width` uses cv2 (less robust, silent fallback). This is a pre-existing helper reused by the worker, so it's not introduced

here ? but the worker now relies on it for training-data correctness where a silent fallback corrupts the manifest. I'll flag it as a major issue with the recommendation to fail loud (or use ffmpeg) when probing for the detector path, since a wrong fps silently mis-aligns every rally label.

Let me do a final check on whether `cmd\_train` detector path leaves the fetched videos around (disk hygiene) ? `videos\_dir` under scratch is never cleaned, and on a real corpus that's many GB. Not a correctness bug but worth a nit. The frame cache is cleaned by the runner; the fetched video copies are not. For S3/GCS that's a real download-to-scratch that accumulates. Minor.

Now let me verify the default synth path truly unchanged one final way: run the existing synth orchestration test (already passed above) ? yes, `test\_cmd\_train\_orchestration` passed. And confirm no `backend` ? `training` import was introduced: the harness is shelled out via subprocess; `synth\_trajectory\_from\_labels` is in `backend.pipeline.detectors.stub\_runner` (backend, not training). Let me grep to be certain no `import training` snuck in.

42. user (2026-06-20T08:59:58.834Z)

```
backend/worker/__main__.py:194:   to training.gen0.harness (backend must NOT import training)
-> push the artifact bundle.
Binary file backend/worker/__pycache__/__main__.cpython-313.pyc matches
```

43. assistant (2026-06-20T09:00:12.193Z)

The only hit is the docstring comment (and the compiled pyc of it). No actual `import training` in backend. The harness is invoked via `subprocess.run([sys.executable, "-m", "training.gen0.harness", ...])`. Constraint satisfied.

Let me confirm the `\_\_getitem\_\_`/`get` interplay once more isn't a problem: `config.get("indexing", {})` returns a dict (verified by the passing stub test `test\_cmd\_train\_detector\_source\_runs\_real\_extract\_via\_stub`, which goes through `\_resolve\_train\_detector` ? `seg.\_resolve\_runner(config.get("indexing", {}))`). That test passed, so the AppConfig?dict?`\_resolve\_runner` path works at runtime.

I have enough to deliver findings. Let me compile the structured output.

44. user (2026-06-20T09:00:44.332Z)

Structured output provided successfully

45. assistant (2026-06-20T09:00:58.150Z)

I completed the adversarial review of the worker real-extract detector path. Summary of what I verified and found:

```
**Verified correct (no bug):**
- The default `synth` path is byte-unchanged from `origin/master`: every label yields a spec (no skips), the pre-loop `>=2` guard already holds, so the new post-loop `len(specs) < 2` guard never fires for synth. `test_cmd_train_orchestration` confirms it.
- `backend` does NOT import `training` ? the gen0 harness is shelled out via `subprocess.run([sys.executable, "-m", "training.gen0.harness", ...])`. The only "training" mention in backend is a docstring.
- `_resolve_train_detector` passing the pydantic `AppConfig` into `seg_cls(None, config)` then `config.get("indexing", {})` works: `AppConfig.get` returns the nested section as a plain dict via `model_dump`, so the segmenter's `idx_cfg.get(...)` calls succeed. The stub detector-source test exercises this end-to-end and passes.
- The manifest contract (`name`, `trajectory`, `fps`, `frame_width`, `labels`) matches what `training/gen0/harness.py` consumes.
- Ruff clean, mypy clean on the changed files; `tests/test_worker.py` + `tests/test_native_wasb_runner.py` = 43 passed.
```

```
**Findings (3 major, 1 minor, 1 nit)** returned via StructuredOutput:
1. **major** ? `_resolve_train_detector` lets `NotImplementedError` from `_build_native_runner` escape uncaught (e.g. `--segmenter tracknet_hybrid` + `detector_impl=native`), crashing
```

`cmd\_train` with a traceback instead of returning exit code 2. Reproduced live.

2. ****major**** ? `video\_by\_stem` indexes `videos/` recursively with no extension filter (labels side filters to `.csv`); a sidecar like `v1.srt` shadows `v1.mp4` on stem collision (last-sorted wins). Reproduced.
3. ****major**** ? `\_probe\_fps\_width` (cv2) silently falls back to 30/1280 on a probe miss; for the REAL detector path this fps feeds the harness's frame?time mapping, so a wrong probe silently mis-aligns every rally label (training-data corruption). `get\_video\_duration` 3 lines earlier already uses robust ffprobe; the fps probe does not.
4. ****minor**** ? fetched videos in `videos\_dir` are never cleaned up (scratch/disk footgun on object-store corpora).
5. ****nit**** ? duplicate `>=2` for LOVO` guards/messages that can drift.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\\_\_main\_\_.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\trajectory\_hybrid.py`.